

Deep Residual Learning for Image Denoising

From Classical Filtering to Channel-Attention ResNets

Adebanji Oluwatimileyin Adelowo

2026

Contents

1	Introduction and Motivation	3
1.1	What this document is	3
1.2	Why denoising matters	4
1.3	Scope of the project studied here	4
2	Problem Definition	5
2.1	The observation model	5
2.2	Denoising as Bayesian estimation	5
2.3	The MMSE estimator, and why the identity skip is principled	6
2.4	Why the problem is ill-posed	7
3	Mathematical and Terminological Foundations	7
3.1	Discrete convolution	7
3.2	Receptive field	8
3.3	Residual learning	9
3.3.1	The residual block	9
3.3.2	Why <i>residual denoising</i> specifically is easier	10
3.4	Batch normalization	10
3.5	Peak signal-to-noise ratio	11
3.6	Structural similarity	12
3.6.1	Construction	12
3.6.2	Properties	13
3.6.3	An implementation subtlety that affects the reported numbers	13
4	Classical Approaches and Why They Plateau	13
4.1	Local averaging: mean and Gaussian filtering	14
4.2	Anisotropic diffusion (Perona and Malik, 1990)	14
4.3	Total variation (Rudin, Osher and Fatemi, 1992)	15
4.4	Wavelet shrinkage (Donoho and Johnstone, 1994)	15
4.5	Non-local means (Buades, Coll and Morel, 2005)	16
4.6	BM3D (Dabov, Foi, Katkovnik and Egiazarian, 2007)	17
4.7	Why the classical family plateaued	17
5	Research Evolution: from Patches to Residual CNNs to Attention	18

5.1	Early neural attempts (2008–2012)	18
5.2	DnCNN (Zhang et al., 2017): the pivot	18
5.3	Residual learning as general infrastructure (He et al., 2016)	19
5.4	Squeeze-and-Excitation (Hu et al., 2018)	19
5.5	RCAN (Zhang et al., 2018): the direct ancestor	19
5.6	Beyond the scope of this project: the current frontier	20
6	The Architecture Used in This Project	21
6.1	Common skeleton	21
6.2	Baseline: plain residual blocks	21
6.3	Squeeze-and-Excitation	22
6.4	The Residual Channel Attention Block as implemented	24
6.5	Fully convolutional inference, and one consequence of it	24
6.6	Summary of the two architectures	25
7	Loss Functions and Optimization	26
7.1	Why MSE blurs	26
7.2	ℓ_1 loss and the conditional median	26
7.3	The SSIM loss	27
7.4	The combined objective used here	27
7.5	Cosine learning-rate annealing	28
7.6	Adam and mixed precision	29
8	Training Methodology	30
8.1	Patch-based sampling	30
8.2	Blind denoising by randomized σ	31
8.3	Data augmentation	32
8.4	Validation, checkpointing, and early stopping	33
9	Code Implementation	33
9.1	Environment and precision (baseline cell 1, enhanced cell 1)	33
9.2	Data acquisition and grayscale conversion (baseline cell 6, enhanced cell 6)	34
9.3	Noise injection: two different regimes	34
9.4	Model construction	35
9.5	Loss and metrics	35
9.6	Training loops	36
9.7	Evaluation code	37
10	Experiments and Results	37
10.1	Dataset	37
10.2	Evaluation protocol	38
10.3	Results	38
10.4	Interpretation: where the 5.39 dB comes from	39
10.5	Discrepancy between the README and the notebook outputs	40
11	Limitations and Failure Cases	40
11.1	Validation and test sets are the same images	40
11.2	The baseline comparison uses a checkpoint from an independent data draw	41
11.3	No ablation, so nothing is attributed	41

11.4	Only one noise level is tested, so the blind claim is untested	42
11.5	Grayscale only	42
11.6	Domain specificity of LFW	42
11.7	No real-noise validation	42
11.8	Metric and protocol caveats	43
11.9	Architectural caveats	43
12	Future Research Directions	43
12.1	The ablation that should be run first	43
12.2	Real-noise datasets and realistic degradation	44
12.3	Self-supervised and unpaired training	44
12.4	Colour, and joint restoration	45
12.5	Modern architectures	45
12.6	Diffusion and plug-and-play	45
12.7	Perceptual objectives and honest metrics	45
12.8	Practical engineering	46
13	Conclusion	46
14	References	46

1 Introduction and Motivation

1.1 What this document is

This document is a technical and research report on the repository `Image_denoising_using_ResNet`. The repository contains two TensorFlow/Keras notebooks that attack the same problem — removing additive Gaussian noise from grayscale face images drawn from the *Labelled Faces in the Wild* (LFW) dataset — at two different levels of architectural and methodological sophistication:

- `final_image_denoising.ipynb` — a **baseline** residual convolutional network: five plain post-activation residual blocks, a global identity skip from input to output, mean-squared-error loss, a single fixed noise level $\sigma = 0.09$, trained on 3,000 full 250×250 images.
- `resnet_enhanced_denoiser.ipynb` — an **enhanced** network: eight Residual Channel Attention Blocks (RCABs) with Squeeze-and-Excitation gating, trained on 80,000 random 64×64 crops with flip augmentation, *blind* to the noise level ($\sigma \sim \mathcal{U}[0.05, 0.15]$, resampled every step), a composite $0.8(1 - \text{SSIM}) + 0.2\ell_1$ objective, and a cosine learning-rate schedule.

The purpose of the document is threefold. First, to derive the mathematics that the code implements, so that every line of the notebooks can be read as an instantiation of a stated estimator, prior, or optimization principle rather than as a stack of library calls. Second, to situate the design inside the research arc that runs from local averaging, through variational and patch-based methods, to convolutional residual learning and attention. Third, to audit the experimental protocol honestly: the results reported here are the ones actually recorded in the stored notebook outputs, and where those disagree with the repository’s `README.md`, or where the protocol contains defects, this is stated explicitly rather than smoothed over.

1.2 Why denoising matters

Noise removal is the oldest and most studied problem in image processing, and it has stayed central for three reasons that have nothing to do with fashion.

It is unavoidable physics. Every digital image is a count of photons. A photosite that collects an expected λ photoelectrons during an exposure actually collects a Poisson random variable with mean and variance λ , so the signal-to-noise ratio of a raw measurement is at best $\sqrt{\lambda}$. Short exposures, small pixels, and low light all reduce λ and therefore degrade SNR. On top of this *shot noise* an image sensor contributes:

- **dark current**, thermally generated electrons accumulating in proportion to exposure time and roughly doubling every 6–8 °C;
- **read noise**, an approximately Gaussian perturbation injected by the source-follower amplifier and the analog front end, largely independent of signal level;
- **fixed-pattern noise**, per-pixel gain (photo-response non-uniformity) and offset variation;
- **quantization noise**, uniform on $[-\frac{1}{2}\Delta, \frac{1}{2}\Delta]$ for a quantizer step Δ , with variance $\Delta^2/12$;
- **amplification**, since raising ISO multiplies the signal *and* the noise already present before the amplifier, so high-ISO images are noisy by construction.

The composite model used in sensor engineering is heteroscedastic Poisson–Gaussian: for a measured value y at a pixel with clean intensity x , $\text{Var}[y] = ax + b$, where a captures the shot component and b the signal-independent read and quantization components. The idealized *additive white Gaussian noise* (AWGN) model with constant σ , which this project uses, is the $a \rightarrow 0$ limit of that model, and it is the standard benchmark setting in the denoising literature.

It is a bottleneck for downstream tasks. Segmentation, registration, detection, recognition, and quantitative measurement all degrade under noise. This is acute in domains where the acquisition dose is deliberately kept low because acquisition is expensive or harmful: low-dose computed tomography, electron and fluorescence microscopy (where photobleaching limits illumination), astronomical imaging of faint sources, and ultrasound. In these domains the denoiser is not a cosmetic post-process; it is what makes the reduced dose acceptable.

It is the canonical inverse problem, and a reusable prior. Denoising is the simplest non-trivial linear inverse problem — the forward operator is the identity — so it isolates the role of the image prior from the role of the forward model. This is why denoising keeps reappearing as a *module* inside other algorithms. Plug-and-Play priors (Venkatakrisnan et al., 2013) and Regularization by Denoising substitute an off-the-shelf denoiser for the proximal operator of an unknown regularizer, solving deblurring, super-resolution, and compressed sensing with a denoiser as the only learned component. Score-based and diffusion generative models (Ho et al., 2020) are, mathematically, stacks of Gaussian denoisers run across a noise-level schedule: their training objective is exactly a denoising regression, and their sampler is a sequence of denoising steps. A better denoiser is therefore a better image prior, with reach far beyond noise removal.

1.3 Scope of the project studied here

The project addresses **single-image, grayscale, synthetic-AWGN, supervised** denoising, with a clean reference available for every training example. That is a deliberately constrained setting. It is the setting in which the field’s methodological lineage (Section 5) is clearest, in which quantitative comparison is unambiguous, and in which a small network can be trained end-to-end on a single

consumer GPU in under two hours. Section 11 is candid about what this setting cannot tell us, particularly about real sensor noise.

2 Problem Definition

2.1 The observation model

Let $x \in \mathbb{R}^{H \times W}$ denote the unknown clean image, understood throughout as a real-valued array normalized to $[0, 1]$, and let $y \in \mathbb{R}^{H \times W}$ denote the observation. The additive white Gaussian noise model states

$$y = x + n, \quad n \sim \mathcal{N}(0, \sigma^2 I_{HW}). \quad (1)$$

Three properties are packed into that one line, and each is an assumption that can fail:

1. **Additivity.** The corruption is independent of the signal. Real shot noise violates this: its variance scales with intensity. Variance-stabilizing transforms such as Anscombe’s map Poisson data approximately onto this model, which is precisely why the AWGN model remains useful even when the physics is Poisson.
2. **Whiteness.** $\mathbb{E}[n_i n_j] = \sigma^2 \delta_{ij}$: the noise has a flat power spectrum and no spatial correlation. Real noise, after demosaicing, colour transformation, and JPEG compression, is strongly spatially correlated, which is the main reason AWGN-trained networks transfer poorly to camera photographs (Plotz and Roth, 2017).
3. **Gaussianity.** The marginal is normal, so the negative log-likelihood is quadratic and the maximum-likelihood data-fidelity term is $\|y - x\|_2^2$.

The repository implements exactly (1), with an additional clipping step analysed in Section 3.5:

```
def add_gaussian_noise(image, sigma):
    """Add zero-mean Gaussian noise with standard deviation sigma."""
    noise = np.random.normal(0, sigma, image.shape)
    return np.clip(image + noise, 0.0, 1.0).astype(np.float32)
```

(*final_image_denoising.ipynb*, cell 4. An identical two-line form appears in the enhanced notebook, cell 5.)

2.2 Denoising as Bayesian estimation

Given y we want x . Under (1) the likelihood is

$$p(y | x) = (2\pi\sigma^2)^{-HW/2} \exp\left(-\frac{\|y - x\|_2^2}{2\sigma^2}\right),$$

so by Bayes’ rule the posterior is $p(x | y) \propto p(y | x)p(x)$, and the *maximum a posteriori* (MAP) estimate is

$$\hat{x}_{\text{MAP}} = \arg \max_x p(x | y) = \arg \min_x \underbrace{\frac{1}{2\sigma^2} \|y - x\|_2^2}_{\text{data fidelity}} + \underbrace{(-\log p(x))}_{\text{regularizer } R(x)}. \quad (2)$$

Equation (2) is the organizing equation of the entire classical literature. Every method in Section 4 is a choice of R :

Method	Implied regularizer $R(x)$
Tikhonov / Gaussian smoothing	$\lambda \ \nabla x\ _2^2$
Total variation (ROF)	$\lambda \ \nabla x\ _1 = \lambda \int \nabla x $
Wavelet soft-thresholding	$\lambda \ Wx\ _1$, W orthonormal
Sparse coding / K-SVD	$\lambda \min_{\alpha} \ \alpha\ _0$ subject to $x \approx D\alpha$
Non-local means / BM3D	implicit: patch self-similarity
CNN denoiser	implicit: encoded in the learned weights

The deep-learning turn can be described in one sentence with respect to (2): instead of *writing down* R and then solving an optimization problem per image, we *learn the minimizer directly* as a function $f_{\theta} : y \mapsto \hat{x}$ by fitting θ on pairs (y_i, x_i) . The prior is never made explicit; it is amortized into the weights, and inference becomes a single forward pass instead of an iterative solve.

2.3 The MMSE estimator, and why the identity skip is principled

MAP is not the only sensible estimator. If we score estimates by squared error, the optimal estimator is the posterior mean:

$$\hat{x}_{\text{MMSE}} = \arg \min_f \mathbb{E}[\|x - f(y)\|_2^2] = \mathbb{E}[x | y]. \quad (3)$$

This matters here for two reasons. First, a network trained with an MSE loss on i.i.d. pairs (x, y) is, in the limit of infinite data and capacity, an approximation of $\mathbb{E}[x | y]$ — which explains the characteristic blur of MSE-trained denoisers (Section 7.1). Second, a classical identity makes the architecture of this project look inevitable.

Tweedie’s formula — attributed to Tweedie by Robbins (1956), stated for the Gaussian location model by Miyasawa (1961), and given a modern treatment by Efron (2011) — says that for the model (1), with $p(y)$ the marginal density of the noisy observation,

$$\mathbb{E}[x | y] = y + \sigma^2 \nabla_y \log p(y). \quad (4)$$

Derivation. Write $p(y) = \int p(x) \mathcal{N}(y; x, \sigma^2 I) dx$. Differentiating under the integral and using $\nabla_y \mathcal{N}(y; x, \sigma^2 I) = \frac{x-y}{\sigma^2} \mathcal{N}(y; x, \sigma^2 I)$ gives

$$\nabla_y p(y) = \frac{1}{\sigma^2} \int (x - y) p(x) \mathcal{N}(y; x, \sigma^2 I) dx.$$

Dividing both sides by $p(y)$ turns the right-hand side into $\frac{1}{\sigma^2} (\mathbb{E}[x | y] - y)$ and the left-hand side into $\nabla_y \log p(y)$. Rearranging gives (4). $\square$

Equation (4) states that **the optimal denoiser is the identity plus a correction**, and that the correction is σ^2 times the score of the noisy marginal. Two consequences follow directly.

- An architecture of the form $\hat{x} = y + g_\theta(y)$ — exactly what both notebooks build with their final `Add()([inp, x])` — is not a convenient training trick. It is the functional form of the Bayes-optimal estimator, with g_θ tasked with learning $\sigma^2 \nabla \log p$. The network is being asked to learn a *score function*, an object whose magnitude is small and whose statistics are far simpler than those of natural images.
- The same identity is the bridge to diffusion models: a network trained to denoise at level σ has, by (4), implicitly learned the score of the data distribution smoothed at scale σ , which is exactly the quantity a score-based sampler needs (Ho et al., 2020; Kadkhodaie and Simoncelli, 2021).

2.4 Why the problem is ill-posed

Hadamard’s conditions for well-posedness require existence, uniqueness, and continuous dependence of the solution on the data. Denoising fails the second outright and the third in practice.

Non-uniqueness. The map $x \mapsto y$ is not injective in any useful sense: for *any* candidate $\tilde{x}$, the residual $y - \tilde{x}$ is a perfectly admissible noise realization, merely one with lower likelihood. The likelihood alone therefore has no useful maximum in x — setting $\hat{x} = y$ gives zero residual and maximal likelihood, which is the useless “do nothing” answer. Formally, $\arg \max_x p(y | x) = y$, and the whole content of denoising sits in R .

Information loss and the fundamental limit. Noise destroys information irreversibly; no estimator can recover what the channel destroyed. Levin and Nadler (2011) estimated the achievable bounds empirically for natural images by non-parametric density estimation over patches, and concluded that at moderate noise levels the best patch-based methods of that era were already within a fraction of a decibel of the limit *for small patch sizes*. That is precisely why classical methods plateaued (Section 4.7), and why subsequent gains came from *enlarging the effective context* rather than from better estimation inside a fixed small patch.

Instability. Even where a unique minimizer of (2) exists, a poorly regularized inverse amplifies perturbations. The practical symptom in learned denoisers is sensitivity to distribution shift: a network trained at one noise level or noise type can behave erratically outside it. This is the direct motivation for the blind training scheme of the enhanced notebook (Section 8.2).

3 Mathematical and Terminological Foundations

3.1 Discrete convolution

For an input feature map $u \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ and a kernel $w \in \mathbb{R}^{k \times k \times C_{\text{in}} \times C_{\text{out}}}$, the layer computed by `Conv2D` (a cross-correlation, as in essentially all deep learning frameworks) is

$$v_{i,j,c'} = b_{c'} + \sum_{c=1}^{C_{\text{in}}} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} w_{p,q,c,c'} u_{i+p-\lfloor k/2 \rfloor, j+q-\lfloor k/2 \rfloor, c}. \quad (5)$$

With `padding='same'` and unit stride, out-of-range indices are read as zero and the spatial size is preserved. Both notebooks use $k = 3$ and `padding='same'` throughout, so every intermediate tensor keeps the input resolution: there is no downsampling path, no pooling, and no decoder. This is the standard design for denoising, where the output must be pixel-aligned with the input and where discarding high-frequency detail through pooling would defeat the purpose.

Three structural properties of (5) carry the inductive bias.

- **Locality.** Each output depends on a $k \times k$ neighbourhood only.
- **Weight sharing.** The same w is applied at every position, making the layer *translation-equivariant*: $f(T_\delta u) = T_\delta f(u)$ for an integer shift T_δ (exactly, away from the boundary). Since the true denoising operator commutes with translation, this is a correct prior, and it is what licenses training on random crops and testing on full images (Section 8.1).
- **Parameter economy.** The layer has

$$P_{\text{conv}} = k^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}} \quad (6)$$

parameters, independent of H and W . For the 3×3 , $64 \rightarrow 64$ convolutions that dominate both models, $P = 9 \cdot 64 \cdot 64 + 64 = 36,928$ — a number that appears verbatim in the Keras summaries stored in both notebooks.

3.2 Receptive field

The *receptive field* of an output unit is the set of input pixels that can influence it. For a stack of layers with kernel sizes k_l and strides s_l , the linear extent obeys the recursion

$$r_l = r_{l-1} + (k_l - 1) \prod_{i=1}^{l-1} s_i, \quad r_0 = 1. \quad (7)$$

With all strides equal to 1 this collapses to $r_L = 1 + \sum_{l=1}^L (k_l - 1)$: every 3×3 layer adds two pixels of diameter, one of radius.

Counting the convolutional layers actually present:

- **Baseline** — one head convolution, five residual blocks with two convolutions each, one tail convolution: $L = 1 + 10 + 1 = 12$ layers of 3×3 , so $r = 1 + 12 \times 2 = 25$, a 25×25 receptive field.
- **Enhanced** — one head, eight RCABs with two 3×3 convolutions each, one tail: $L = 1 + 16 + 1 = 18$, so $r = 1 + 18 \times 2 = 37$, a 37×37 receptive field.

The 1×1 convolutions inside the Squeeze-and-Excitation branch add nothing to r by (7), but the global average pooling that precedes them aggregates over the *entire* feature map. The attention path therefore has an unbounded (image-wide) receptive field, even though the residual path has only 37 pixels. This asymmetry is the main representational contribution of channel attention in this network, and it has a subtle training/inference consequence discussed in Section 6.5.

Receptive field is the right lens for the classical-to-deep transition. NLM and BM3D search a 21×21 or 39×39 window for similar patches, so their effective context is comparable to a shallow CNN. Depth is how CNNs buy context cheaply: r grows linearly in depth while parameter count also grows only linearly, whereas a single convolution achieving $r = 37$ would need $37^2 = 1369$ taps per channel pair.

3.3 Residual learning

3.3.1 The residual block

Let $\mathcal{H}(u)$ be the mapping a block is supposed to realize. He et al. (2016) proposed to parameterize instead the *residual* $\mathcal{F}(u) = \mathcal{H}(u) - u$ and to recover the target by addition:

$$v = \mathcal{F}(u, \{W_i\}) + u. \quad (8)$$

The reformulation is exactly equivalent in expressive power — any $\mathcal{H}$ is representable if $\mathcal{F}$ is — but it is not equivalent in *optimization difficulty*.

Identity is free. If the optimal $\mathcal{H}$ is close to the identity, the optimal $\mathcal{F}$ is close to zero. Standard initializers put weights near zero and standard regularizers pull them toward zero, so the network *starts* near a good solution and must learn only a small perturbation. In the plain parameterization the network would have to approximate the identity with a stack of nonlinear layers, which is difficult in a way that is easy to underestimate. He et al. (2016) demonstrated this with the *degradation problem*: a 56-layer plain network had **higher training** error than a 20-layer one — not an overfitting effect but an optimization failure, which residual connections removed.

Gradients flow. Consider a chain of blocks with identity shortcuts, $v_{l+1} = v_l + \mathcal{F}(v_l, W_l)$. Unrolling from layer l to layer L ,

$$v_L = v_l + \sum_{i=l}^{L-1} \mathcal{F}(v_i, W_i),$$

and therefore, by the chain rule,

$$\frac{\partial \mathcal{L}}{\partial v_l} = \frac{\partial \mathcal{L}}{\partial v_L} \left(1 + \frac{\partial}{\partial v_l} \sum_{i=l}^{L-1} \mathcal{F}(v_i, W_i) \right). \quad (9)$$

The constant 1 is the whole point. Gradient reaches layer l *undiminished* regardless of depth, because the derivative decomposes into a direct term plus a correction. Vanishing would require the summed Jacobian to equal exactly -1 across a mini-batch, which is vanishingly unlikely. In a plain network the same derivative is a *product* of per-layer Jacobians, whose spectral norms multiply, giving the familiar exponential vanishing or explosion. He et al. (2016b) made this precise and showed that the clean form of (9) requires a genuinely *unmodulated* identity path — scaling, gating, or placing a nonlinearity on the shortcut degrades it. Note that both notebooks use the original ResNet-v1 ordering, with a ReLU applied *after* the addition:

```
x = Add()([shortcut, x])
x = Activation('relu')(x)
```

so their skip path is not perfectly clean in the He et al. (2016b) sense. At the depths used here (5 and 8 blocks) this is immaterial; at 50 or more blocks it would not be.

3.3.2 Why *residual denoising* specifically is easier

Beyond the generic optimization argument, denoising has a domain-specific reason to prefer the residual form. Under (1), $n = y - x$; so if a network is given y and outputs $\hat{n}$, the reconstruction is $\hat{x} = y - \hat{n}$ and the regression target is **the noise**, not the image. Compare the two targets:

Property	Target = x (direct)	Target = n (residual)
Mean	image-dependent, ≈ 0.5	exactly 0
Dynamic range	full $[0, 1]$	concentrated in $[-3\sigma, 3\sigma]$
Spatial structure	edges, texture, faces, long-range correlation	i.i.d., no structure
Power spectrum	approximately $1/f^2$, heavy low-frequency content	flat (white)
State at initialization	must be synthesized from scratch	low frequencies already correct

The direct parameterization forces the network to spend capacity *re-synthesizing the low-frequency content that is already present in the input undamaged*. AWGN perturbs every frequency equally, but natural images have almost all their energy at low frequencies; so at the pixel level the input is already an excellent estimate of the output, and the residual parameterization lets the network devote itself to the high-frequency band where the local SNR is actually poor. This is the argument Zhang et al. (2017) made when introducing DnCNN, and it is why DnCNN predicts $\hat{n}$ rather than $\hat{x}$. It is also, of course, Tweedie’s formula (4) again, arrived at from a signal-processing rather than a Bayesian direction.

Convention used in this repository. Both notebooks apply the global skip as an *addition*:

```
x = Conv2D(1, (3, 3), padding='same',
           kernel_initializer='he_normal', dtype='float32')(x)
out = Add(dtype='float32')([inp, x])
```

so the tail convolution learns $r_\theta(y) \approx x - y = -n$, i.e. the *negative* noise. This is equivalent to DnCNN’s $\hat{n}$ formulation up to a sign absorbed into the last layer’s weights; nothing of substance differs.

3.4 Batch normalization

For a mini-batch $\mathcal{B}$ and a given channel, batch normalization (Ioffe and Szegedy, 2015) computes

$$\begin{aligned}\mu_{\mathcal{B}} &= \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} u_i, & s_{\mathcal{B}}^2 &= \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (u_i - \mu_{\mathcal{B}})^2, \\ \hat{u}_i &= \frac{u_i - \mu_{\mathcal{B}}}{\sqrt{s_{\mathcal{B}}^2 + \varepsilon}}, & \text{BN}(u_i) &= \gamma \hat{u}_i + \beta,\end{aligned}\tag{10}$$

where for convolutional inputs the statistics are pooled over batch *and* spatial positions, so a 64-channel feature map contributes 2×64 learnable parameters (γ, β) and 2×64 non-trainable running estimates used at inference. This accounting is directly visible in the stored model summaries: each

BatchNormalization layer reports $256 = 4 \times 64$ parameters, of which 128 are trainable and 128 are not.

The original justification was reduction of “internal covariate shift”; the more durable explanation is that BN reparameterizes the loss surface so that it is smoother and its gradients more predictive, permitting larger learning rates (Santurkar et al., 2018). It also decouples the scale of a layer’s weights from its function, since $\text{BN}(\alpha Wu) = \text{BN}(Wu)$ for $\alpha > 0$.

A caveat specific to restoration. BN is standard in classification but has repeatedly been found *harmful* in super-resolution and restoration. Lim et al. (2017) removed it from EDSR and reported both better accuracy and substantially lower memory use, reasoning that normalizing features destroys the absolute intensity and contrast information a restoration network must preserve, and that range flexibility matters when the output lives in the same space as the input. RCAN (Zhang et al., 2018) likewise contains no BN, and NAFNet (Chen et al., 2022) uses LayerNorm rather than BN. This repository retains BN in both models — a defensible choice for a small network trained from scratch at a large initial learning rate, but a deviation from the practice of the very architectures it takes as ancestors, and a natural first ablation (Section 12.1).

3.5 Peak signal-to-noise ratio

PSNR is a monotone reparameterization of mean squared error onto a logarithmic scale. For images with maximum representable value L ,

$$\text{MSE}(x, \hat{x}) = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W (x_{ij} - \hat{x}_{ij})^2, \quad \text{PSNR} = 10 \log_{10} \frac{L^2}{\text{MSE}}. \quad (11)$$

With the $[0, 1]$ normalization used throughout the repository, $L = 1$ and the implementation reduces to $\text{PSNR} = -10 \log_{10} \text{MSE}$:

```
def PSNR(orig, pred):
    """Peak Signal-to-Noise Ratio (dB). Higher is better."""
    mse = np.mean((orig.astype(np.float64) - pred.astype(np.float64)) ** 2)
    if mse == 0:
        return float('inf')
    return 10.0 * np.log10(1.0 / mse)
```

(*final_image_denoising.ipynb*, cell 9.)

Interpretation. A gain of $10 \log_{10} 2 \approx 3.01$ dB corresponds to halving the MSE; 6.02 dB to quartering it. Equivalently, a gain of Δ dB divides the residual error energy by $10^{\Delta/10}$.

The predicted PSNR of the noisy input, and why the measurement exceeds it. For an unclipped observation, $\mathbb{E}[\text{MSE}(x, y)] = \sigma^2$, so

$$\mathbb{E}[\text{PSNR}] \approx -20 \log_{10} \sigma = -20 \log_{10}(0.09) = 20.92 \text{ dB}.$$

The notebook, however, records 21.42 dB for the noisy input. The discrepancy is not sampling error; it is the clipping. Let $C = [0, 1]^{HW}$, a closed convex set, and let P_C denote Euclidean projection onto it — which is precisely what `np.clip` computes elementwise. Projections onto convex sets

are non-expansive, hence $\|P_C(y) - P_C(x)\|_2 \leq \|y - x\|_2$. Because the clean image already satisfies $x \in C$, we have $P_C(x) = x$, so

$$\text{MSE}(x, \text{clip}(y)) \leq \text{MSE}(x, y) \implies \text{PSNR}_{\text{clipped}} \geq 20.92 \text{ dB}.$$

Clipping is therefore a mild denoiser in its own right, guaranteed never to hurt, and the observed +0.50 dB is fully accounted for. The same argument applies to the `np.clip(..., 0, 1)` applied to every network prediction before evaluation.

What PSNR does not capture. PSNR is a pixelwise, signal-independent measure. It is invariant to spatial rearrangement of the error, so a small uniform blur and a structured artifact of the same energy score identically, even though a human observer ranks them very differently. It correlates only weakly with subjective quality across distortion types, a point argued at length by Wang and Bovik (2009). This is the motivation for SSIM.

3.6 Structural similarity

3.6.1 Construction

Wang et al. (2004) built SSIM from the premise that the human visual system is adapted to extract *structural* information, and that quality should therefore be assessed by how much structure survives, with luminance and contrast — which are scene-illumination properties, not structure — factored out.

For two aligned local windows a and b (vectors of N pixels), define the local statistics

$$\mu_a = \frac{1}{N} \sum_i a_i, \quad \sigma_a^2 = \frac{1}{N-1} \sum_i (a_i - \mu_a)^2, \quad \sigma_{ab} = \frac{1}{N-1} \sum_i (a_i - \mu_a)(b_i - \mu_b),$$

and three comparison functions:

$$l(a, b) = \frac{2\mu_a\mu_b + C_1}{\mu_a^2 + \mu_b^2 + C_1} \quad (\text{luminance}), \tag{12}$$

$$c(a, b) = \frac{2\sigma_a\sigma_b + C_2}{\sigma_a^2 + \sigma_b^2 + C_2} \quad (\text{contrast}), \tag{13}$$

$$s(a, b) = \frac{\sigma_{ab} + C_3}{\sigma_a\sigma_b + C_3} \quad (\text{structure}). \tag{14}$$

Each satisfies three axioms: symmetry, $l(a, b) = l(b, a)$; boundedness by 1; and a unique maximum, attained only when the compared quantity is equal. The stabilizing constants prevent division by near-zero denominators and are set as $C_1 = (K_1 L)^2$ and $C_2 = (K_2 L)^2$ with $K_1 = 0.01$, $K_2 = 0.03$ by convention.

Note what (14) is: s is exactly the Pearson correlation coefficient between the two windows. That is the sense in which SSIM measures “structure” — it is the cosine of the angle between the mean-removed, contrast-normalized patch vectors, so it is invariant to any affine change of intensity, which is precisely the invariance the luminance and contrast terms are there to measure separately.

The general index is $\text{SSIM} = l^\alpha c^\beta s^\gamma$. Setting $\alpha = \beta = \gamma = 1$ and $C_3 = C_2/2$ collapses the product:

$$c(a, b) s(a, b) = \frac{2\sigma_a\sigma_b + C_2}{\sigma_a^2 + \sigma_b^2 + C_2} \cdot \frac{\sigma_{ab} + C_2/2}{\sigma_a\sigma_b + C_2/2} = \frac{2\sigma_a\sigma_b + C_2}{\sigma_a^2 + \sigma_b^2 + C_2} \cdot \frac{2\sigma_{ab} + C_2}{2\sigma_a\sigma_b + C_2} = \frac{2\sigma_{ab} + C_2}{\sigma_a^2 + \sigma_b^2 + C_2},$$

giving the familiar closed form

$$\text{SSIM}(a, b) = \frac{(2\mu_a\mu_b + C_1)(2\sigma_{ab} + C_2)}{(\mu_a^2 + \mu_b^2 + C_1)(\sigma_a^2 + \sigma_b^2 + C_2)}. \quad (15)$$

3.6.2 Properties

- **Symmetry:** $\text{SSIM}(a, b) = \text{SSIM}(b, a)$.
- **Boundedness:** $\text{SSIM}(a, b) \leq 1$, with equality if and only if $a = b$. For the structure term alone the range is $[-1, 1]$; anticorrelated windows score negative.
- **Not a metric:** $1 - \text{SSIM}$ does not satisfy the triangle inequality in general, so it is a similarity index rather than a distance.
- **Locality:** (15) is evaluated on a sliding window and the *mean SSIM* over all window positions is reported. Wang et al. (2004) recommend an 11×11 circularly symmetric Gaussian window with $\sigma_w = 1.5$ normalized to unit sum, so that μ , σ^2 and σ_{ab} above become weighted moments. The Gaussian weighting avoids the blocking artifacts a uniform window produces in the SSIM map.

3.6.3 An implementation subtlety that affects the reported numbers

The repository computes SSIM in two different places with two different window definitions.

- **In the training loss** (enhanced notebook, cell 9) it calls `tf.image.ssim(y_true, y_pred, max_val=1.0)`, which implements Wang et al. (2004) faithfully: an 11×11 Gaussian window with $\sigma_w = 1.5$, $K_1 = 0.01$, $K_2 = 0.03$.
- **In the reported evaluation metric** (both notebooks) it calls `skimage.metrics.structural_similarity(..., data_range=1.0)` with otherwise default arguments. The `scikit-image` default is `gaussian_weights=False` with a 7×7 **uniform** window and a sample (unbiased) covariance estimate — *not* the configuration of Wang et al. (2004). Obtaining the canonical index requires passing `gaussian_weights=True`, `sigma=1.5` and `use_sample_covariance=False` explicitly.

The two give systematically different values on the same image pair, and the difference is not negligible at high noise. This does not invalidate the internal comparison of Section 10 — all three rows of the results table use the same `skimage` call, so the *ranking* is sound — but it does mean that the absolute SSIM values are not directly comparable with published numbers, and that the quantity optimized during training is not exactly the quantity reported. The point recurs in Section 11.

4 Classical Approaches and Why They Plateau

This section is not a historical courtesy. Each classical family encodes an assumption about images that the convolutional network of Section 6 also encodes, but implicitly and more flexibly; reading them in order makes the network’s inductive biases legible.

4.1 Local averaging: mean and Gaussian filtering

The oldest denoiser replaces each pixel by a weighted average of its neighbourhood, $\hat{x} = h * y$ with a non-negative kernel h summing to one. The bias–variance calculus is immediate. For a flat region of true value c ,

$$\mathbb{E}[\hat{x}_i] = c \quad (\text{unbiased}), \quad \text{Var}[\hat{x}_i] = \sigma^2 \sum_j h_j^2.$$

For a uniform $m \times m$ box, $\sum_j h_j^2 = 1/m^2$, so the noise standard deviation falls by a factor of m : a 3×3 mean filter buys $20 \log_{10} 3 \approx 9.5$ dB *in flat regions*. The price is paid at every place where the region is not flat. Expanding around a pixel, the bias of a symmetric kernel with variance σ_h^2 is

$$\mathbb{E}[\hat{x}_i] - x_i \approx \frac{1}{2} \sigma_h^2 \Delta x_i,$$

proportional to the Laplacian — that is, largest exactly at edges and fine texture. This is the fundamental tension of all local denoising: variance reduction requires averaging, averaging across a discontinuity produces bias, and a *spatially invariant* kernel cannot tell the two situations apart.

The Gaussian kernel $h_\tau(u) = (2\pi\tau)^{-1} \exp(-\|u\|^2/2\tau)$ is distinguished because convolving with it is exactly the solution of the isotropic heat equation

$$\frac{\partial u}{\partial t} = \Delta u, \quad u(\cdot, 0) = y,$$

evaluated at $t = \tau/2$. Denoising is thus diffusion, and the failure mode is that heat diffuses across edges as readily as along them. Everything that follows is an attempt to make the diffusion, or the averaging, *content-aware*.

4.2 Anisotropic diffusion (Perona and Malik, 1990)

Perona and Malik (1990) replaced the constant diffusivity with a decreasing function of the local gradient magnitude:

$$\frac{\partial u}{\partial t} = \text{div}(g(\|\nabla u\|) \nabla u), \tag{16}$$

with, for example,

$$g(s) = \exp(-(s/K)^2) \quad \text{or} \quad g(s) = \frac{1}{1 + (s/K)^2}.$$

Where the gradient is small (flat region) $g \approx 1$ and (16) behaves like the heat equation, smoothing aggressively; where the gradient is large (an edge) $g \approx 0$ and diffusion is switched off, so the edge is preserved. Decomposing the divergence in the local gauge coordinates (η, ξ) aligned with and orthogonal to the gradient makes the mechanism explicit:

$$\frac{\partial u}{\partial t} = g(s) u_{\xi\xi} + (g(s) + s g'(s)) u_{\eta\eta}, \quad s = \|\nabla u\|,$$

where $u_{\xi\xi}$ is the second derivative *along* the edge and $u_{\eta\eta}$ *across* it. Diffusion along the edge always has non-negative coefficient $g(s) \geq 0$. The coefficient across the edge, $g(s) + s g'(s)$, becomes *negative* for s larger than a threshold, so the process runs the heat equation *backwards* across strong edges — it sharpens them. This is why Perona–Malik enhances edges rather than merely preserving them, and it is also why the continuous formulation is ill-posed: backward diffusion is unstable, and the scheme is stabilized only by the discretization and by regularizing the gradient estimate. The practical artifact is *staircasing*, the conversion of smooth ramps into piecewise-constant plateaus.

4.3 Total variation (Rudin, Osher and Fatemi, 1992)

The ROF model states denoising as a variational problem in the form of (2):

$$\hat{x} = \arg \min_u \frac{1}{2} \|u - y\|_2^2 + \lambda \text{TV}(u), \quad \text{TV}(u) = \int_{\Omega} \|\nabla u\| \, du. \quad (17)$$

The choice of the ℓ_1 norm of the gradient, rather than the ℓ_2 norm of Tikhonov regularization, is the entire idea. A quadratic penalty $\int \|\nabla u\|^2$ charges a step of height h spread over k pixels a cost of order h^2/k , so it is always cheaper to spread the step — quadratic regularization *cannot* produce sharp edges. The total variation of the same step is h regardless of k : TV is indifferent to how sharply the transition is made, so discontinuities survive. Formally, functions of bounded variation admit jump discontinuities while functions in H^1 (finite Dirichlet energy) do not.

The Euler–Lagrange equation of (17),

$$u - y - \lambda \operatorname{div} \left(\frac{\nabla u}{\|\nabla u\|} \right) = 0,$$

exposes both the strength and the weakness: the divergence term is curvature motion, so level sets are smoothed by their curvature, and the diffusivity $1/\|\nabla u\|$ degenerates where the gradient vanishes. TV therefore also staircases, and it penalizes texture — which has large total variation but is signal, not noise — as heavily as it penalizes noise. Modern variants (total generalized variation, higher-order and structure-tensor models) exist precisely to soften these two failures.

4.4 Wavelet shrinkage (Donoho and Johnstone, 1994)

Let W be an orthonormal wavelet transform (Mallat, 1989). Applying it to (1),

$$\underbrace{Wy}_w = \underbrace{Wx}_\theta + \underbrace{Wn}_\epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2 I),$$

where the crucial fact is that **orthonormal transforms preserve white Gaussian noise**: if $n \sim \mathcal{N}(0, \sigma^2 I)$ then $Wn \sim \mathcal{N}(0, \sigma^2 WW^\top) = \mathcal{N}(0, \sigma^2 I)$. The noise is therefore spread uniformly over all coefficients, while a piecewise-smooth image is *sparse* in the wavelet basis: a few large coefficients on edges, and near-zero coefficients elsewhere. Signal and noise separate by magnitude, and denoising reduces to coefficient-wise thresholding.

Donoho and Johnstone (1994) analysed this and proposed *soft thresholding*,

$$\eta_T(w) = \operatorname{sign}(w) \max(|w| - T, 0), \quad (18)$$

with the *universal threshold* $T = \sigma\sqrt{2\log N}$ for N coefficients — chosen because the maximum of N i.i.d. standard normals concentrates near $\sqrt{2\log N}$, so with high probability *every* pure-noise coefficient is killed. They proved that the resulting estimator is within a $\log N$ factor of the *ideal* estimator that knows which coefficients carry signal, uniformly over a wide class of smoothness spaces — a remarkable near-minimax guarantee.

Soft thresholding is not an ad hoc rule. It is exactly the proximal operator of the ℓ_1 norm:

$$\eta_T(w) = \arg \min_u \frac{1}{2}(u - w)^2 + T|u|,$$

as is checked by subdifferentiating. Wavelet shrinkage is therefore MAP estimation (2) under a Laplacian prior on wavelet coefficients, solved in closed form because the orthonormal basis decouples the coordinates. Hard thresholding ($\eta_T(w) = w \cdot \mathbf{1}[|w| > T]$) preserves large coefficients exactly but is discontinuous, giving visible ringing; soft thresholding is continuous but biases large coefficients toward zero by T .

The residual artifact of any critically sampled wavelet shrinkage is *pseudo-Gibbs ringing* near edges, caused by the lack of translation invariance of the transform; cycle-spinning (averaging over shifts) mitigates it at the cost of redundancy.

4.5 Non-local means (Buades, Coll and Morel, 2005)

Every method above is *local*: it uses a pixel’s spatial neighbourhood. Buades et al. (2005) observed that natural images are highly *self-similar* — a patch on one edge of a face resembles patches on other similar edges elsewhere — and that similar patches provide independent noise realizations of essentially the same underlying signal. Averaging them therefore reduces variance *without* blurring, because the averaged samples were never spatially adjacent in the first place.

Define a patch neighbourhood $\mathcal{N}_i$ around pixel i and a Gaussian-weighted patch distance $d(i, j) = \|y(\mathcal{N}_i) - y(\mathcal{N}_j)\|_{2,a}^2$. Then

$$\hat{x}_i = \sum_{j \in \Omega} w(i, j) y_j, \quad w(i, j) = \frac{1}{Z(i)} \exp\left(-\frac{d(i, j)}{h^2}\right), \quad (19)$$

with $Z(i)$ the normalizing constant making the weights sum to one, and h a filtering parameter controlling the decay of the weights.

A refinement that appears in the authors’ extended treatment (Buades et al., 2005b) and in most practical implementations replaces $d(i, j)$ in the exponent by $\max(d(i, j) - 2\sigma^2, 0)$. The reason is that two patches whose *underlying* signal is identical still differ by $2\sigma^2$ in expectation, because each carries its own independent noise; subtracting that offset prevents genuinely matching patches from being systematically down-weighted, and matters most at high σ .

Two remarks make the connection to the rest of this document. First, the weights in (19) are a softmax over patch similarities — structurally the same operation as the attention mechanism of a transformer, with patch inner products in place of query–key products. Non-local means is, in a precise sense, self-attention over pixels, invented thirteen years before transformers reached vision. Second, the weights are *computed from the noisy image itself*, which makes NLM adaptive without any training, but also makes the weights noisy at high σ , which is where NLM degrades.

The cost is the obvious drawback: for a search window of $S \times S$ and a patch of $P \times P$, the complexity is $O(HWS^2P^2)$, which is why practical implementations restrict the search to a window rather than the whole image.

4.6 BM3D (Dabov, Foi, Katkovnik and Egiazarian, 2007)

BM3D is the high-water mark of the classical era, and it works by *combining* every idea above: non-local grouping, transform-domain sparsity, and shrinkage. It runs in two passes.

Step 1 — hard-thresholding estimate.

1. *Grouping.* For each reference block, find the K most similar blocks by ℓ_2 distance within a search window, and stack them into a 3-D array (a “group”). Because the blocks are similar, this array is highly correlated along the third (stacking) axis.
2. *Collaborative filtering.* Apply a separable 3-D decorrelating transform: a 2-D transform (DCT or biorthogonal wavelet) within each block, and a 1-D transform (typically Haar) along the stacking axis. The added third dimension is where the gain comes from — the group is far sparser in 3-D than any single block is in 2-D, because the inter-block correlation is now concentrated into the DC coefficient of the 1-D transform.
3. Hard-threshold the 3-D spectrum, invert the transform, and obtain an estimate for every block in the group.
4. *Aggregation.* Each pixel belongs to many overlapping groups; combine the several estimates by a weighted average whose weights are inversely proportional to the number of retained coefficients (i.e. to the estimated variance of each estimate).

Step 2 — Wiener filtering. Repeat grouping using the *step-1 estimate* for block matching (more reliable, since it is already denoised), and replace hard thresholding with an empirical Wiener filter that shrinks each 3-D coefficient by

$$W = \frac{|\tau(\hat{x}^{\text{step1}})|^2}{|\tau(\hat{x}^{\text{step1}})|^2 + \sigma^2},$$

the optimal linear shrinkage given the step-1 spectrum as an estimate of the true signal energy. Aggregate as before.

BM3D held the state of the art for roughly a decade and remains the standard non-learned baseline.

4.7 Why the classical family plateaued

By around 2010, successive papers were improving on BM3D by hundredths of a decibel. The reasons are structural, not incidental.

- **Hand-designed transforms are a fixed, narrow model class.** DCT and wavelets are excellent for piecewise-smooth signals with point and (with curvelets) curve singularities. They are mediocre for texture, and they encode nothing about faces, or about any other semantic regularity. A learned filter bank can adapt to whatever regularity is actually present in the data.
- **The prior is fixed, not learned from data.** BM3D’s parameters are tuned once, globally. It cannot exploit the fact that a corpus of aligned frontal faces is a far smaller and more structured set than “all natural images” — precisely the exploitation that lets a small CNN reach 33.7 dB on LFW.

- **Estimation is per-image and myopic.** Each image is processed in isolation with no memory of the millions of images seen before. All the statistical strength must be extracted from self-similarity within a single image, which is a weak signal at high noise where the patch-matching itself is corrupted.
- **The near-optimality result cuts both ways.** Levin and Nadler (2011) showed the best patch-based methods were close to optimal *for small patch sizes*, and that the bound improves with patch size — but the number of examples needed to estimate a non-parametric patch prior grows exponentially with patch dimension. Classical methods had no way to spend more context; a deep network does, because depth converts context into parameters linearly rather than exponentially.
- **The pipelines are non-differentiable and hard to compose.** BM3D’s block matching involves discrete selection, so its stages cannot be jointly optimized end-to-end for a downstream criterion, and no perceptual objective can be back-propagated through it.

5 Research Evolution: from Patches to Residual CNNs to Attention

5.1 Early neural attempts (2008–2012)

Jain and Seung (2008) trained a small convolutional network for denoising and argued that CNNs implement a form of approximate inference in a Markov random field while being much cheaper at test time. The result was competitive but not decisively better than the best classical methods of its day. Burger et al. (2012) then showed that a plain multi-layer perceptron applied to 17×17 patches, trained on millions of examples, could *match and slightly exceed* BM3D. Their conclusion was not architectural but empirical: with enough training data and capacity, a generic learned mapping beats a carefully hand-designed algorithm. The two limitations were that the MLP was patch-based (no weight sharing across positions, so parameters scaled badly) and that a separate network had to be trained for each noise level.

Chen and Pock (2017) took the complementary route with *trainable nonlinear reaction diffusion*: unrolling a fixed number of diffusion steps (cf. (16)) into a feed-forward network and learning the filters and the influence functions by back-propagation. This “unrolled optimization” viewpoint retains the interpretability of the variational model while gaining the performance of learning, and it remains an influential design pattern.

5.2 DnCNN (Zhang et al., 2017): the pivot

DnCNN is the paper this repository’s baseline most directly descends from, and its three contributions map one-to-one onto design decisions in the notebooks.

1. **Residual learning of the noise.** Rather than outputting $\hat{x}$, DnCNN outputs $\hat{n}$ and the clean estimate is $y - \hat{n}$; the loss is $\frac{1}{2N} \sum_i \|\mathcal{R}(y_i; \theta) - (y_i - x_i)\|_F^2$. The argument is the one made in Section 3.3.2. In the notebooks this appears as the global `Add()([inp, x]) skip`.
2. **Batch normalization inside every layer**, which Zhang et al. found *synergistic* with residual learning specifically for denoising: the residual target is approximately Gaussian, and BN keeps the intermediate distributions stable enough that a large learning rate can be used. In the notebooks this appears as `BatchNormalization()` in every block.
3. **A single blind model (DnCNN-B) for a range of noise levels.** Training one network on $\sigma \in [0, 55]$ (on a 0–255 scale) produced a model competitive with, and often better than,

the individually trained noise-specific models — and, remarkably, also capable of handling super-resolution and JPEG deblocking when trained on a mixture of degradations. In the notebooks this appears as the enhanced model’s $\sigma \sim \mathcal{U}[0.05, 0.15]$ sampling.

DnCNN outperformed BM3D by 0.5–0.7 dB at typical noise levels with a plain 17-layer 3×3 CNN — no non-local operations, no explicit patch matching, no hand-designed transform. It is the moment the classical plateau broke.

The immediate descendants extended it in the obvious directions. FFDNet (Zhang et al., 2018b) concatenated a *noise-level map* to the input, turning σ into a controllable input rather than a hidden assumption and giving a single network explicit control over the denoising strength, plus a downsampling strategy for speed. CBDNet (Guo et al., 2019) added a noise-estimation subnetwork and a realistic in-camera noise model in order to attack real photographs rather than synthetic AWGN.

5.3 Residual learning as general infrastructure (He et al., 2016)

The residual block itself came from image classification. He et al. (2016) introduced it to solve the degradation problem described in Section 3.3.1, and trained networks of 152 layers (and, as a demonstration, 1202) where previously depth beyond about 20 layers had *hurt training* error. He et al. (2016b) then analysed *why*, showing that a clean identity path — no scaling, no gating, no post-addition nonlinearity — yields the additive gradient decomposition of (9) and proposing the pre-activation ordering (BN–ReLU–Conv–BN–ReLU–Conv) that makes it exact. Residual connections are now infrastructure rather than a technique: they appear in essentially every subsequent architecture, transformers included.

For restoration specifically, Lim et al. (2017) adapted the block by *removing* batch normalization (Section 3.4) and adding constant residual scaling for stability at large widths, producing EDSR and winning the NTIRE 2017 super-resolution challenge.

5.4 Squeeze-and-Excitation (Hu et al., 2018)

Convolution mixes information across channels with a *fixed* linear map: the weight $w_{p,q,c,c'}$ in (5) does not depend on the input. Hu et al. (2018) proposed making the per-channel contribution *input-dependent* by computing a global descriptor of each channel and using it to gate that channel. This is cheap — $2C^2/r$ parameters for a reduction ratio r , no spatial parameters at all — and it gave consistent accuracy gains across architectures, winning the ILSVRC 2017 classification task. The full mathematics is developed in Section 6.3, since it is what the enhanced model implements.

The conceptual point is that SE is *attention over the channel axis*. Where spatial attention asks “which locations matter”, channel attention asks “which feature detectors matter for this particular input”, and it answers using information pooled from the whole image.

5.5 RCAN (Zhang et al., 2018): the direct ancestor

Zhang et al. (2018) combined the two ideas explicitly for super-resolution, introducing the **Residual Channel Attention Block (RCAB)** — the block whose name and structure the enhanced notebook borrows. Their motivation was restoration-specific and worth stating precisely: in super-resolution and denoising, low-frequency information passes through the network essentially unchanged and is therefore *abundant and redundant*, while the high-frequency information that ac-

tually needs recovering is scarce. Treating all channels equally wastes capacity on the redundant part. Channel attention lets the network allocate representational effort adaptively.

RCAN’s architecture has two levels of residual nesting — *residual in residual* (RIR): residual groups each containing several RCABs, with long skip connections around each group and around the whole trunk. This lets the long skips carry the low frequencies directly to the output while the deep trunk concentrates on the residual detail. RCAN reached over 400 layers, far deeper than anything before it in super-resolution.

How the notebook’s RCAB differs from Zhang et al.’s. This is worth tabulating, because the repository’s block shares the name but not the exact structure.

Aspect	RCAN (Zhang et al., 2018)	This repository (enhanced)
Block body	Conv $\rightarrow$ ReLU $\rightarrow$ Conv	Conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Conv $\rightarrow$ BN
Normalization	none	BatchNorm after both convolutions
Attention	channel attention, reduction $r = 16$	Squeeze-and-Excitation, reduction $r = 8$
After the skip addition	nothing (clean identity)	ReLU
Macro structure	residual-in-residual: 10 groups $\times$ 20 blocks, with long skips	flat stack of 8 blocks, one global skip
Depth	> 400 conv layers	18 conv layers
Pooling in attention	global average pool	global average pool

The differences are all in the direction of a smaller, more conservative network, which is appropriate for the scale of this project; but they mean the block should be described as *SE-augmented ResNet-v1*, which is what it is, rather than as a reimplementaion of RCAN.

5.6 Beyond the scope of this project: the current frontier

The architecture studied here sits at roughly the 2018 state of the art. For orientation, the trajectory since then has three main strands.

Transformers. Self-attention supplies exactly what convolution lacks — a content-dependent, long-range mixing operator — but naive self-attention costs $O((HW)^2)$, which is intolerable at image resolution. IPT (Chen et al., 2021) applied a large pre-trained transformer to patches for multiple restoration tasks. SwinIR (Liang et al., 2021) adopted shifted-window attention to make the cost linear in the number of pixels. **Restormer** (Zamir et al., 2022) made the sharpest move for restoration specifically: it computes attention across the *channel* dimension rather than the spatial one, so the attention map is $C \times C$ instead of $HW \times HW$, giving global spatial context at linear cost. Restormer’s multi-Dconv head transposed attention is, viewed from this document’s angle, a full-rank content-adaptive generalization of exactly the Squeeze-and-Excitation gate in Section 6.3 — SE computes a diagonal channel-modulation matrix from a globally pooled descriptor; Restormer computes a dense one from spatially resolved projections.

Simplification. **NAFNet** (Chen et al., 2022) pushed in the opposite direction, asking how much of the machinery is load-bearing. The answer was: remarkably little. Replacing nonlinear activations with a *SimpleGate* (splitting the channels in half and multiplying the halves elementwise)

and attention with a simplified channel attention, they achieved 40.30 dB on the SIDD real-noise benchmark at less than half the computational cost of the previous state of the art. The lesson is that the multiplicative gating pattern — which SE introduced — matters more than the specific nonlinearities layered around it.

Generative and diffusion denoisers. By Tweedie’s formula (4), a denoiser *is* a score estimator, so denoising and generative modelling are the same problem viewed from two directions. Diffusion models (Ho et al., 2020) exploit this to sample from the image prior; conversely, Kadkhodaie and Simoncelli (2021) showed that the prior implicit in a trained denoiser can be used to solve arbitrary linear inverse problems by a denoiser-driven stochastic ascent. These methods can produce outputs that are *perceptually* far superior to a regression denoiser — they sample from $p(x | y)$ rather than returning its mean — at the cost of lower PSNR, which is the perception–distortion trade-off (Blau and Michaeli, 2018) discussed in Section 7.1.

6 The Architecture Used in This Project

6.1 Common skeleton

Both notebooks build the same overall shape, differing only in the block used and in the number of blocks:

```
input (H x W x 1)
|
|-- Conv 3x3, 1 -> 64 -> ReLU [head]
|
|-- Block_1 -> Block_2 -> ... -> Block_N [trunk]
|
|-- Conv 3x3, 64 -> 1 (float32) [tail]
|
+-- Add(input, tail_output) (float32) [global residual skip]
|
output (H x W x 1)
```

The head lifts the single-channel image into a 64-dimensional feature space; the trunk does the work; the tail projects back to one channel and produces the *residual* estimate; the global skip adds the input back, giving the $\hat{x} = y + g_\theta(y)$ form justified in Section 2.3.

Two implementation details are shared. First, both models run under `mixed_float16` precision, so activations are half-precision while the master weights stay in float32; the tail convolution and the final `Add` are explicitly pinned to `dtype='float32'` so the residual addition and the subsequent loss computation do not lose precision where the values are smallest. Second, all convolutions use `he_normal` initialization, i.e. $w \sim \mathcal{N}(0, 2/\text{fan_in})$, the variance-preserving choice for ReLU networks (He et al., 2015): a ReLU zeroes half its inputs, so the factor 2 compensates for the halved variance of the output.

6.2 Baseline: plain residual blocks

```
def res_block(x, num_channels):
    """Residual block: Conv->BN->ReLU->Conv->BN -> add skip -> ReLU."""
```

```

shortcut = x
x = Conv2D(num_channels, (3, 3), padding='same',
           kernel_initializer='he_normal')(x)
x = BatchNormalization()(x)
x = Activation('relu')(x)
x = Conv2D(num_channels, (3, 3), padding='same',
           kernel_initializer='he_normal')(x)
x = BatchNormalization()(x)
x = Add()([shortcut, x])
x = Activation('relu')(x)
return x

```

(*final_image_denoising.ipynb*, cell 11.)

In the notation of (8) with $C = 64$ channels,

$$\mathcal{F}(u) = \text{BN}_2\left(W_2 * \text{ReLU}(\text{BN}_1(W_1 * u))\right), \quad v = \text{ReLU}(u + \mathcal{F}(u)). \quad (20)$$

Parameter accounting. Per block, using (6) and the BN accounting of Section 3.4:

$$\underbrace{36,928}_{W_1} + \underbrace{256}_{\text{BN}_1} + \underbrace{36,928}_{W_2} + \underbrace{256}_{\text{BN}_2} = 74,368.$$

With five blocks, plus a head of $9 \cdot 1 \cdot 64 + 64 = 640$ and a tail of $9 \cdot 64 \cdot 1 + 1 = 577$:

$$5 \times 74,368 + 640 + 577 = 373,057,$$

which matches the stored `model.summary()` output exactly (373,057 total; 371,777 trainable; 1,280 non-trainable, the latter being $5 \times 2 \times 128$ BN running statistics).

6.3 Squeeze-and-Excitation

Let $u \in \mathbb{R}^{H \times W \times C}$ be the output of the block body. SE proceeds in three stages.

Squeeze. Collapse each channel to a single scalar by global average pooling, producing a channel descriptor $z \in \mathbb{R}^C$:

$$z_c = F_{\text{sq}}(u_c) = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W u_{i,j,c}. \quad (21)$$

The purpose is to escape the receptive-field limitation: every convolution in the trunk sees at most a 37×37 neighbourhood, but z_c summarizes the channel over the whole image. It is the cheapest possible global context aggregator — a first-order statistic per channel, with no parameters.

Excitation. Map the descriptor to a gate through a two-layer bottleneck with a ReLU between and a sigmoid at the end:

$$s = F_{\text{ex}}(z) = \sigma(W_2 \delta(W_1 z)), \quad W_1 \in \mathbb{R}^{\frac{C}{r} \times C}, \quad W_2 \in \mathbb{R}^{C \times \frac{C}{r}}, \quad (22)$$

where δ is ReLU and σ the logistic sigmoid. Each design choice carries weight:

- **The bottleneck** ($r = 8$ here, 16 in Hu et al.). Without it, a full $C \times C$ map would cost $C^2 = 4096$ parameters *per block*; the bottleneck costs $2C^2/r = 2 \cdot 4096/8 = 1024$. Beyond economy, the rank constraint is a regularizer: it forces the gate to be a function of a low-dimensional summary of the channel activations rather than allowing arbitrary per-channel memorization.
- **The nonlinearity between the two layers** is what makes the gate more than a linear reweighting; without δ , $W_2 W_1$ would collapse to a single rank- C/r linear map.
- **The sigmoid, rather than a softmax.** A softmax would force the gates to compete and sum to one, i.e. it would allow only *relative* emphasis. The sigmoid gates channels *independently*, so several channels can be emphasized or suppressed simultaneously. This is the rationale Hu et al. give explicitly: the sigmoid is chosen to learn a *non-mutually-exclusive* relationship between channels, so that multiple channels may be emphasized rather than one winning at the others' expense.

Rescale. Modulate the block body channel-wise:

$$\tilde{u}_{:,c} = s_c \cdot u_{:,c}. \quad (23)$$

Two properties worth drawing out.

The gate can only attenuate. Since $s_c \in (0, 1)$, (23) scales the residual branch down, never up. SE therefore doubles as a learned, content-adaptive version of the constant residual scaling that Szegedy et al. (2017) and Lim et al. (2017) introduced by hand to stabilize very wide residual networks. The network can learn to make a block nearly a no-op on inputs where it is unhelpful.

It introduces a genuinely new function class. The composition in (23) is *multiplicative* in the input: s depends on u , so the map $u \mapsto s(u) \odot u$ is quadratic-like, and its Jacobian $\partial \tilde{u} / \partial u$ depends on u itself. No stack of convolutions and ReLUs computes this cheaply — a ReLU network is piecewise *linear* in its input, whereas the SE gate is smoothly input-modulated. This is the same mechanism that makes NAFNet's SimpleGate effective and, in more general form, what attention provides in a transformer.

The implementation uses 1×1 convolutions instead of **Dense** layers, which is mathematically identical once the descriptor has been reshaped to $1 \times 1 \times C$ (a 1×1 convolution on a 1×1 spatial map *is* a fully connected layer), but keeps everything in the convolutional dtype and avoids a flatten/reshape round trip:

```
def se_block(x, channels, ratio=8):
    """Squeeze-and-Excitation: globally re-weight channels by learned
    importance."""
    s = GlobalAveragePooling2D()(x) # (B, C)
    s = Reshape((1, 1, channels))(s)
    s = Conv2D(max(1, channels // ratio), 1, use_bias=False,
activation='relu')(s)
    s = Conv2D(channels, 1, use_bias=False, activation='sigmoid')(s)
```

```
return Multiply()([x, s])
```

(*resnet_enhanced_denoiser.ipynb*, cell 10.)

Note `use_bias=False` on both projections, so the parameter count is exactly $C \cdot (C/r) + (C/r) \cdot C = 64 \cdot 8 + 8 \cdot 64 = 1024$, with no bias terms — confirmed below against the stored summary.

6.4 The Residual Channel Attention Block as implemented

```
def rcab(x, channels):
    """Residual Channel Attention Block: Conv-BN-ReLU-Conv-BN-SE-Add-ReLU."""
    shortcut = x
    x = Conv2D(channels, 3, padding='same', kernel_initializer='he_normal')(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    x = Conv2D(channels, 3, padding='same', kernel_initializer='he_normal')(x)
    x = BatchNormalization()(x)
    x = se_block(x, channels)
    x = Add()([shortcut, x])
    x = Activation('relu')(x)
    return x
```

(*resnet_enhanced_denoiser.ipynb*, cell 10.)

Combining (20), (21), (22) and (23), the block computes

$$\mathcal{F}(u) = \text{BN}_2\left(W_2 * \text{ReLU}(\text{BN}_1(W_1 * u))\right), \quad v = \text{ReLU}\left(u + F_{\text{ex}}(F_{\text{sq}}(\mathcal{F}(u))) \odot \mathcal{F}(u)\right), \quad (24)$$

where $\odot$ denotes channel-wise (broadcast) multiplication. The single difference from (20) is the gate applied to $\mathcal{F}(u)$ before the addition — note that it gates the *residual branch only*, never the identity path, which preserves the gradient argument of (9).

Parameter accounting. Per RCAB: 74,368 (as for the baseline block) + 1,024 (SE) = 75,392. With eight blocks, plus head and tail:

$$8 \times 75,392 + 640 + 577 = 604,353,$$

matching the stored summary exactly (604,353 total; 602,305 trainable; 2,048 = $8 \times 2 \times 128$ non-trainable BN statistics). The enhanced model is therefore $1.62\times$ the size of the baseline, of which the attention machinery accounts for only $8 \times 1024 = 8,192$ parameters, i.e. **1.4% of the total**. Whatever the enhanced model gains, it does not gain by spending parameters on attention.

6.5 Fully convolutional inference, and one consequence of it

```
def build_enhanced_model(height=None, width=None, channels=64, n_blocks=8):
    """
```


Fully convolutional - trains on 64x64 patches, infers on any size.

```
"""
inp = Input(shape=(height, width, 1))
```

(resnet_enhanced_denoiser.ipynb, cell 10.)

The enhanced model declares `Input(shape=(None, None, 1))`, so the same weights train on 64×64 patches and run on 250×250 images. This is sound for the convolutional path, by the translation-equivariance argument of Section 3.1: a convolution’s behaviour at a pixel depends only on its neighbourhood, not on the image size. The baseline, by contrast, hard-codes `build_model(250, 250, ...)`, which is unnecessary but harmless.

There is, however, a genuine train/test mismatch introduced by the SE branch, and it is worth stating because it is easy to miss. The squeeze operation (21) averages over HW positions. At training time $HW = 64^2 = 4096$; at test time $HW = 250^2 = 62,500$, a $15.3\times$ larger sample. The *expectation* of z_c is unchanged — crops are unbiased samples of image content — but its *variance* is not: for a channel with spatial variance v_c and spatial correlation length ℓ , $\text{Var}[z_c]$ scales roughly as $v_c \ell^2 / (HW)$. The descriptor distribution seen at inference is therefore substantially more concentrated than the one the excitation MLP was trained on, so the gates s_c are pushed toward their central values and the attention becomes systematically *less* discriminative on full images than on patches. A further, second-order effect is that a 64×64 crop of a face may contain only skin, or only an eye, so its channel statistics are far more varied across the training set than whole-face statistics are.

This is not fatal — the measured full-image results in Section 10 are strong — but it predicts that the attention mechanism contributes less at test resolution than the patch-level validation loss suggests, and it is a concrete candidate explanation to test in an ablation. Mitigations used in the literature include training on larger crops, and evaluating with tiled inference at the training patch size.

6.6 Summary of the two architectures

	Baseline	Enhanced
Blocks	5 plain residual	8 RCAB (residual + SE)
Conv layers (3×3)	12	18
Width	64	64
Receptive field (conv path)	25×25	37×37
Receptive field (attention path)	—	whole image
Total parameters	373,057	604,353
Trainable	371,777	602,305
Non-trainable (BN statistics)	1,280	2,048
Attention parameters	0	8,192 (1.4%)
Input shape	fixed $250 \times 250 \times 1$	(None, None, 1)
Precision policy	<code>mixed_float16</code>	<code>mixed_float16</code>
Global residual skip	yes (float32)	yes (float32)

7 Loss Functions and Optimization

7.1 Why MSE blurs

The baseline compiles with `loss='mean_squared_error'`. The empirical risk is

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(y_i) - x_i\|_2^2, \quad (25)$$

which is the sample version of $\mathbb{E}\|f_{\theta}(y) - x\|^2$. By (3), its population minimizer over all measurable f is the posterior mean $\mathbb{E}[x | y]$. That fact is the entire explanation of MSE-induced blur, and it is worth spelling out concretely.

Suppose a noisy patch y is genuinely ambiguous: given the noise, the true edge could plausibly lie at column j or at column $j + 1$, with posterior probability $\frac{1}{2}$ each. The two hypotheses are both sharp images. The posterior mean is their average, which is an edge with a two-pixel ramp — an image that is *not a sample from the posterior at all*, and indeed may have probability zero under the image prior. The MSE-optimal answer is a hedge, and hedging over spatial hypotheses is exactly what “blur” means. The effect is strongest where the posterior is most multimodal — fine texture, hair, skin pores — which is precisely the content a viewer inspects to judge sharpness.

Formally: let $q(\cdot) = p(x | y)$. Then $\mathbb{E}_q\|x - \hat{x}\|^2 = \text{tr Cov}_q[x] + \|\mathbb{E}_q[x] - \hat{x}\|^2$, so the squared-error criterion is minimized by the mean regardless of how implausible that mean is as an image. Any estimator that returns a plausible *sample* instead necessarily has higher expected squared error — this is the distortion side of the perception–distortion trade-off proved by Blau and Michaeli (2018): improving perceptual quality (measured as divergence between the output distribution and the natural-image distribution) *must* increase distortion, at least beyond a certain point. The trade-off is not an artifact of imperfect training; it is a property of the objectives.

Two further, more mundane problems with MSE:

- **It is dominated by large errors.** The quadratic penalty weights an error of 10ε a hundred times more than an error of ε , so gradients are dominated by the hardest pixels (occlusion boundaries, specularities) rather than by the mass of the image.
- **It is spatially and structurally blind.** (25) is a sum of independent per-pixel terms; permuting the error field leaves it unchanged. It has no way to express “this error destroyed an edge” versus “this error shifted the mean brightness slightly”.

7.2 ℓ_1 loss and the conditional median

The natural first fix is to change the exponent. The population minimizer of $\mathbb{E}|x - f(y)|$ is the conditional *median*, not the mean. Medians are robust to the tails of the posterior, and in the two-hypothesis example above the median is one of the two sharp edges rather than their average — so ℓ_1 has a structurally weaker pull toward blur.

The gradient structure differs too. For MSE the per-pixel gradient is $2(\hat{x} - x)$, proportional to the error, so pixels that are already nearly correct contribute almost nothing; for ℓ_1 it is $\text{sign}(\hat{x} - x)$, of constant magnitude, so small residual errors keep receiving full-strength gradient until they are resolved. Zhao et al. (2017) evaluated this empirically across denoising, demosaicing and super-resolution and reported a genuinely striking result: the network trained with the ℓ_1 loss achieved

a *lower* ℓ_2 error than the network trained with the ℓ_2 loss itself. Since ℓ_2 is by construction the objective matched to that metric, the effect cannot be an objective-matching artifact; the authors attribute it to convergence behaviour, hypothesizing that ℓ_2 is more readily trapped in poor local minima while ℓ_1 reaches better solutions under *both* metrics — an explanation consistent with the constant-magnitude gradient described above.

7.3 The SSIM loss

Since SSIM is differentiable in $\hat{x}$ almost everywhere (it is a smooth rational function of local moments, which are linear in the image), it can be used directly as a training objective. Because higher is better, one minimizes

$$\mathcal{L}_{\text{SSIM}}(x, \hat{x}) = 1 - \text{SSIM}(x, \hat{x}), \quad (26)$$

with SSIM the mean over sliding windows as in Section 3.6. Zhao et al. (2017) derived the analytic gradient and noted a property that matters in practice: the derivative of the SSIM loss at a pixel depends on the *local statistics of the window around it*, so the effective per-pixel learning rate is content-adaptive — larger where the local contrast is low (a region where errors are perceptually conspicuous) and smaller in high-variance texture. MSE has no such modulation.

What the SSIM term contributes structurally, reading off (15):

- the luminance factor l penalizes local mean shift, i.e. low-frequency drift;
- the contrast factor c penalizes *loss of local variance* — and this is the key term for denoising, because over-smoothing reduces $\sigma_{\hat{x}}$ and is therefore directly punished, in a way that MSE never punishes it;
- the structure factor s , being a correlation coefficient, penalizes any change in the *pattern* of the local window, independent of its scale and offset.

An SSIM-only objective has a complementary weakness: because l , c and s are all invariant to certain global transformations of the window, the loss is comparatively insensitive to uniform intensity or contrast errors. A network trained on SSIM alone can drift in absolute brightness. This is exactly the gap that an ℓ_1 term closes.

7.4 The combined objective used here

```
def combined_loss(y_true, y_pred):
    """0.8 x (1-SSIM) + 0.2 x L1. Both terms cast to float32 for stability."""
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.cast(y_pred, tf.float32)
    l1 = tf.reduce_mean(tf.abs(y_true - y_pred))
    ssim = tf.reduce_mean(tf.image.ssim(y_true, y_pred, max_val=1.0))
    return 0.2 * l1 + 0.8 * (1.0 - ssim)
```

(resnet_enhanced_denoiser.ipynb, cell 9.)

That is

$$\mathcal{L} = \alpha (1 - \text{SSIM}(x, \hat{x})) + (1 - \alpha) \|x - \hat{x}\|_1, \quad \alpha = 0.8. \quad (27)$$

The division of labour is clean: the SSIM term supplies *structural and contrast* fidelity, and prevents the over-smoothing that a pure regression objective invites; the ℓ_1 term supplies *absolute pixel* fidelity, anchoring the output’s brightness and preventing the drift that SSIM alone permits. Zhao et al. (2017) proposed and validated exactly this mixture, in their case $\mathcal{L}^{\text{Mix}} = \alpha \mathcal{L}^{\text{MS-SSIM}} + (1 - \alpha) G_{\sigma_G} \cdot \mathcal{L}^{\ell_1}$, where the ℓ_1 term is additionally weighted by the Gaussian window used in the MS-SSIM computation. They set $\alpha = 0.84$ empirically, chosen so that the contributions of the two losses would be roughly balanced. The repository’s $\alpha = 0.8$ is therefore close to the published value, with two differences worth noting: this project uses single-scale SSIM rather than multi-scale, and it applies a plain unweighted mean for the ℓ_1 term rather than the Gaussian-weighted one. The mixture has since become a common default in restoration.

How the two terms actually balance in this run. The mixing weights are not directly interpretable, because the two terms have different natural scales. We can recover the balance from the stored training log. At epoch 5 the notebook records `val_loss: 0.08456` and `val_mae_metric: 0.0283`. Since the reported metric is exactly the ℓ_1 term, the decomposition is

$$\underbrace{0.2 \times 0.0283}_{\ell_1 \text{ contribution} = 0.00566} + \underbrace{0.8(1 - \text{SSIM})}_{= 0.08456 - 0.00566 = 0.0789},$$

so at that point $1 - \text{SSIM} = 0.0789/0.8 = 0.0986$, i.e. the validation SSIM on 64×64 patches was already ≈ 0.901 . More importantly, the SSIM term accounts for $0.0789/0.08456 = 93.3\%$ of the loss value, and therefore dominates the gradient. The nominal 80/20 split is in practice closer to 93/7. This is not a defect — the ℓ_1 term is doing its job as an anchor, not as a co-equal objective — but it should be understood correctly: the model is, to a good approximation, an SSIM-trained model with a small pixel-fidelity regularizer.

A note on why this is not simply “better than MSE” by fiat. Optimizing SSIM and reporting SSIM is a form of matched objective; part of the enhanced model’s SSIM advantage in Section 10 is attributable to training on the metric it is scored by. The PSNR advantage is the more informative number, because the enhanced model is *not* trained on MSE and still wins by a wide margin on PSNR.

7.5 Cosine learning-rate annealing

The baseline uses a fixed learning rate of 10^{-3} with a reactive `ReduceLROnPlateau` (halve on 3 epochs without improvement, floor 10^{-6}). The enhanced model uses a scheduled cosine decay (Loshchilov and Hutter, 2017, introduced there as the decay phase of SGDR):

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T}\right), \quad t = 0, \dots, T. \quad (28)$$

The shape is the point. Compared with a step schedule, (28) spends a long initial stretch near $\eta_{\max}$ — because $\cos$ is flat near 0 — which favours exploration of the loss landscape and, with BN, keeps the implicit regularization of large-step SGD active. It then decays smoothly with no discontinuities (a step drop injects a transient that can destabilize Adam’s second-moment estimate), and finally flattens again near $\eta_{\min}$, giving many small refinement steps in the basin the optimizer has settled into. There are no hyperparameters to tune beyond the endpoints and T .

```

EPOCHS = 60
total_steps = EPOCHS * steps_per_epoch

lr_schedule = keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=1e-3,
    decay_steps=total_steps,
    alpha=1e-6,          # final LR
)

```

(*resnet_enhanced_denoiser.ipynb*, cell 11.)

A discrepancy worth recording. Keras implements

$$\text{decayed}(t) = (1 - \alpha) \cdot \frac{1}{2} \left(1 + \cos \frac{\pi t}{T} \right) + \alpha, \quad \eta_t = \eta_0 \cdot \text{decayed}(t),$$

and documents `alpha` as “*minimum learning rate value for decay as a fraction of initial_learning_rate*”. With `initial_learning_rate=1e-3` and `alpha=1e-6`, the floor is therefore $\eta_{\min} = 10^{-3} \times 10^{-6} = 10^{-9}$, not the 10^{-6} that the inline comment and the `README.md` both claim. To obtain a floor of 10^{-6} the argument should be `alpha=1e-3`. In practice this has little effect — by the time the schedule is within a factor of 10^{-3} of its initial value the updates are already negligible, and `EarlyStopping(patience=8)` means the schedule’s tail may never be reached at all, since $T = 60 \times 1000 = 60,000$ steps assumes all 60 epochs run. But the documented behaviour and the actual behaviour differ, and a reader reproducing the work should know it.

7.6 Adam and mixed precision

Both notebooks optimize with Adam (Kingma and Ba, 2015):

$$\begin{aligned}
 m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, & v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\
 \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, & \hat{v}_t &= \frac{v_t}{1 - \beta_2^t}, & \theta_t &= \theta_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}},
 \end{aligned}$$

with Keras defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-7}$. The per-parameter normalization by $\sqrt{\hat{v}_t}$ makes Adam largely insensitive to gradient scale, which matters here because the residual-learning target is small in magnitude and because the two loss terms in (27) have very different scales.

Both notebooks set `mixed_precision.set_global_policy('mixed_float16')`. Under this policy, forward and backward computation runs in float16 (roughly doubling throughput on tensor-core GPUs and halving activation memory, which is what makes the 250\$×\$250 baseline trainable at batch 32 at all), while a float32 copy of the weights is maintained for the update, since a float16 update of magnitude 10^{-7} relative to a weight of magnitude 10^{-1} would simply be rounded away. Keras additionally wraps the optimizer in a dynamic `LossScaleOptimizer`, multiplying the loss by a large factor before back-propagation so that small gradient magnitudes do not underflow float16’s minimum normal of about 6×10^{-5} , and dividing it back out before the update. The notebooks correctly pin the output convolution, the residual `Add`, and the loss computation to float32, which are the three places where precision actually matters.

8 Training Methodology

8.1 Patch-based sampling

```
PATCH_SIZE    = 64
PATCHES_EACH = 16  # patches per image -> 5000 x 16 = 80 000 patches

def extract_patches(imgs, patch_size=PATCH_SIZE, n=PATCHES_EACH, seed=42):
    rng = np.random.default_rng(seed)
    H, W = imgs.shape[1], imgs.shape[2]
    patches = []
    for img in imgs:
        for _ in range(n):
            r = rng.integers(0, H - patch_size)
            c = rng.integers(0, W - patch_size)
            patches.append(img[r:r+patch_size, c:c+patch_size, :])
    return np.array(patches, dtype=np.float32)
```

(*resnet_enhanced_denoiser.ipynb*, cell 7. Stored output: All patches: (80000, 64, 64, 1).)

There are four distinct reasons this is the right move, and one cost.

1. It is licensed by translation equivariance. As established in Section 3.1, a fully convolutional network computes a *local* operator: the value at an output pixel depends only on the 37×37 neighbourhood around it, not on the image size or on the pixel’s absolute position. Training that operator on crops and applying it to full images is therefore not an approximation — for pixels whose receptive field lies wholly inside the crop it is exact.

2. Activation memory, not parameter memory, is the binding constraint. The model has under 10^6 parameters, which is nothing; what fills the GPU is the stored activations needed for back-propagation. Per sample, the trunk holds on the order of 40 tensors of shape $H \times W \times 64$. In float16:

per sample, full image: $250^2 \cdot 64 \cdot 2 \text{ B} \approx 8.0 \text{ MB}$ per tensor,

per sample, patch: $64^2 \cdot 64 \cdot 2 \text{ B} \approx 0.52 \text{ MB}$ per tensor.

The ratio is $250^2/64^2 = 15.3$. This is what lets the enhanced model use a *deeper* network (18 conv layers versus 12) at a *larger* batch (64 versus 32) on the same hardware.

3. It multiplies the number of gradient signals per epoch. The baseline sees 3,000 training examples, giving 85 optimization steps per epoch at batch 32. The enhanced model sees 64,000 patches, giving 1,000 steps per epoch at batch 64 — $11.8\times$ more parameter updates per epoch, from the same underlying 5,000 photographs. Because SGD-family convergence is governed by the number of *steps* rather than the number of *epochs*, this alone is a large practical advantage.

4. It decorrelates the mini-batch. A batch of 32 full LFW portraits is highly redundant: they are all aligned frontal faces with the same coarse layout, so their gradients point in similar directions. A batch of 64 random crops contains a mixture of skin, hair, eye, background and edge

content at random positions, which gives a lower-variance-per-unit-compute estimate of the true gradient direction.

The cost: boundary statistics. With `padding='same'`, output pixels within 18 pixels of the border (the receptive-field radius, $\lfloor 37/2 \rfloor$) see zero-padding rather than real image content. The fraction of pixels with a *fully valid* receptive field is

$$\text{patch: } \frac{(64 - 36)^2}{64^2} = \frac{784}{4096} = 19.1\%, \quad \text{full image: } \frac{(250 - 36)^2}{250^2} = \frac{45,796}{62,500} = 73.3\%.$$

So during training, four out of five supervised pixels are boundary-affected, whereas at test time only one in four is. The network devotes a large share of its capacity to the padded-boundary regime, which is over-represented relative to deployment. Standard remedies are to compute the loss only on the valid interior of each patch, or to use a larger patch. This is a genuine (if second-order) methodological weakness of the enhanced setup, and it compounds with the global average pooling mismatch of Section 6.5 — both are consequences of training at one resolution and testing at another.

8.2 Blind denoising by randomized σ

```
BATCH = 64
SIGMA_MIN, SIGMA_MAX = 0.05, 0.15

def augment_and_noise(patch):
    """Random flip + random Gaussian noise (blind denoising)."""
    patch = tf.image.random_flip_left_right(patch)
    patch = tf.image.random_flip_up_down(patch)
    sigma = tf.random.uniform(), SIGMA_MIN, SIGMA_MAX, dtype=tf.float32)
    noisy = tf.clip_by_value(patch + tf.random.normal(tf.shape(patch),
stddev=sigma), 0.0, 1.0)
    return tf.cast(noisy, tf.float16), tf.cast(patch, tf.float16)
```

(*resnet_enhanced_denoiser.ipynb*, cell 8.)

Instead of a single σ , the training distribution becomes

$$\sigma \sim \mathcal{U}[0.05, 0.15], \quad y = \text{clip}(x + n), \quad n \sim \mathcal{N}(0, \sigma^2 I),$$

with a *fresh* σ and a *fresh* noise realization drawn for every patch on every step. Three things follow.

The network must estimate the noise level internally. With σ fixed, a network can bake the correct shrinkage amount into its weights. With σ random and unobserved, the optimal estimator is $\mathbb{E}[x | y] = \int \mathbb{E}[x | y, \sigma] p(\sigma | y) d\sigma$, so the network must infer $p(\sigma | y)$ from the input itself. This is feasible: the local variance in flat regions is an unbiased estimate of σ^2 , and robust estimators such as the median absolute deviation of the finest wavelet subband do exactly this classically. The SE branch is well suited to carrying this information, since a globally pooled channel descriptor is precisely a global statistic of the sort a noise-level estimate requires — which offers a mechanistic hypothesis for why attention and blind training combine well. This is the mechanism Zhang et

al. (2017) exploited in DnCNN-B and made explicit in FFDNet (Zhang et al., 2018b) by feeding a noise-level map directly.

One model covers a range. Practically, a single set of weights serves any σ in the trained range, with graceful degradation outside it, instead of requiring a model zoo indexed by noise level. The test point used in Section 10, $\sigma = 0.09$, sits near the middle of $[0.05, 0.15]$ (whose mean is 0.10), so the model is evaluated in the well-covered interior of its training distribution. Note the asymmetry this creates in the comparison: the *baseline* is trained at exactly the test σ and thus enjoys a specialist’s advantage, while the enhanced model is a generalist tested at one point. The enhanced model nevertheless wins by 5.4 dB.

It regularizes. Randomizing σ is data augmentation in the degradation space. It prevents the network from over-fitting to the exact shrinkage profile of one noise level, and empirically it improves robustness to mild misspecification of the noise model.

The single most consequential difference from the baseline, however, is not σ but *when the noise is drawn*. The baseline generates its noisy training set once, before training:

```
SIGMA = 0.09
images_noisy = add_gaussian_noise(images_clean, SIGMA)
```

(*final_image_denoising.ipynb*, cell 7.)

Every epoch therefore presents the network with the *same* (y_i, x_i) pairs, including the same frozen noise realization n_i . Over 50 epochs a network with 3.7×10^5 parameters and only 3,000 examples can begin to memorize those specific realizations — learning n_i rather than learning to remove noise — which shows up as a widening train/validation gap. The enhanced pipeline draws new noise inside `tf.data` on every step, so each patch is seen with a different corruption every time it is visited. Statistically, the enhanced model is trained on i.i.d. draws from the true joint $p(x, y)$, i.e. on an *effectively infinite* dataset in y , whereas the baseline is trained on a fixed finite sample of it. This is a textbook variance-reduction argument and is likely the largest single contributor to the gap in Section 10.

8.3 Data augmentation

The two flips generate the group $\{e, \text{flip}_h, \text{flip}_v, \text{flip}_h \circ \text{flip}_v\}$, which is the Klein four-group — a subgroup of the dihedral group D_4 of the square, omitting the four 90° rotations. Since the true denoising operator commutes with every isometry of the pixel grid,

$$f^*(g \cdot y) = g \cdot f^*(x), \quad g \in D_4,$$

augmenting by g imposes an *equivariance* prior: it tells the network that these four transformations should not change its behaviour, which shrinks the effective hypothesis space by up to a factor of four without adding parameters. Combined with the 16 random crops per image, the nominal 80,000 patches become 320,000 distinct inputs, and combined with continuous noise resampling, effectively unbounded.

A remark on the vertical flip. Horizontal flipping of a face yields another plausible face — faces are approximately bilaterally symmetric, and LFW is pose-normalized — so it is squarely on the data manifold. Vertical flipping yields an upside-down face, which is *off* the manifold of LFW at the semantic level. This is nevertheless harmless and arguably beneficial for denoising specifically:

the operator being learned is local and low-level, its receptive field (37 px) is far smaller than a face, and forcing the network to work on inverted faces discourages it from relying on face-level semantic priors that would not transfer. For a *generative* face model the same augmentation would be a mistake.

The 90° rotations are not used, which leaves a factor of two on the table at zero cost; adding `tf.image.rot90` with a random k would complete the D_4 orbit. The same eight transformations are also the standard basis of *self-ensembling* at test time (average the predictions over the eight transformed inputs, un-transformed), which typically buys 0.1–0.2 dB for free and is not done here.

8.4 Validation, checkpointing, and early stopping

The baseline holds out 10% of its 3,000 training images via `validation_split=0.1` and uses three callbacks: `EarlyStopping` with `patience=6` and `restore_best_weights=True`; `ReduceLROnPlateau` with `factor=0.5`, `patience=3` and `min_lr=1e-6`; and `ModelCheckpoint` writing `best_denoiser.keras` with `save_best_only=True`.

The enhanced model splits its patches 80/20 and uses `EarlyStopping` with `patience=8` and `restore_best_weights=True`, plus `ModelCheckpoint`. Its validation pairs are generated at a *fixed* $\sigma = 0.09$ rather than a random one:

```
def make_val_pair(patch):
    sigma = 0.09
    noisy = tf.clip_by_value(patch + tf.random.normal(tf.shape(patch),
stddev=sigma), 0.0, 1.0)
    return tf.cast(noisy, tf.float16), tf.cast(patch, tf.float16)
```

(*resnet_enhanced_denoiser.ipynb*, cell 8.)

Fixing σ for validation is the right call — it removes one source of variance from the model-selection signal — but note that it makes the validation criterion a measurement at a single noise level, so early stopping optimizes for $\sigma = 0.09$ specifically even though the model is nominally blind. It also means the validation loss is *not* an estimate of the training objective’s expectation, only of its value at one point of the σ distribution.

A more serious issue with this validation set is identified in Section 11.1.

9 Code Implementation

This section walks the two notebooks side by side, connecting each block of real code to the mathematics above. Cell numbers refer to the zero-indexed cell order in the committed `.ipynb` files.

9.1 Environment and precision (baseline cell 1, enhanced cell 1)

```
%tensorflow_version 2.x
import tensorflow as tf

# Enable memory growth so the GPU allocates only what it needs
gpus = tf.config.list_physical_devices('GPU')
```

```

if gpus:
    for gpu in gpus:
        tf.config.experimental.set_memory_growth(gpu, True)

# Mixed precision: use float16 on GPU, float32 for master weights
from tensorflow.keras import mixed_precision
mixed_precision.set_global_policy('mixed_float16')

```

Both notebooks open identically. The stored outputs confirm Compute dtype: float16 / Variable dtype: float32 and a T4-class /device:GPU:0. The rationale is Section 7.6.

9.2 Data acquisition and grayscale conversion (baseline cell 6, enhanced cell 6)

```

ds = dataset.shuffle(7000).take(5000)
images_clean = np.array([d["image"] for d in tfds.as_numpy(ds)])
images_clean = color.rgb2gray(images_clean).astype(np.float32)
images_clean = images_clean[:, :, :, np.newaxis]
print("Clean images:", images_clean.shape)

```

Both notebooks draw 5,000 of LFW’s 13,233 images, convert RGB to luminance using `skimage.color.rgb2gray` — i.e. the ITU-R BT.709 luma $Y = 0.2125R + 0.7154G + 0.0721B$, which also rescales to $[0, 1]$ — and append a singleton channel axis. Stored output: (5000, 250, 250, 1).

Two consequences. First, everything downstream is single-channel, so no cross-channel noise correlation can be exploited (Section 11.5). Second, and less obviously, the grayscale conversion *itself* denoises slightly: a weighted average of three colour channels whose noise is partially independent reduces noise variance. Since the “clean” targets are produced by the same conversion, this does not bias the experiment, but it does mean the clean reference is a processed quantity, not a ground truth in the physical sense.

A more important caveat, which the notebooks do not address. LFW images are JPEG-compressed news photographs. The arrays called `images_clean` therefore already contain real camera noise, demosaicing artifacts, and 8×8 DCT blocking. The experiment measures the network’s ability to invert *synthetic AWGN added on top of an already-degraded reference*, and PSNR is computed against that imperfect reference. This inflates apparent performance relative to a truly clean ground truth, and it means the network is implicitly trained to reproduce JPEG artifacts faithfully.

9.3 Noise injection: two different regimes

Baseline (cells 4, 7, 8). Noise is materialized once, into a second array of the same size as the dataset, and then split:

```

SIGMA = 0.09
images_noisy = add_gaussian_noise(images_clean, SIGMA)

train_noisy = images_noisy[:3000]
train_clean = images_clean[:3000]

```

```
test_noisy = images_noisy[3000:]
test_clean = images_clean[3000:]
```

Stored output: Train: (3000, 250, 250, 1), Test: (2000, 250, 250, 1). The memory cost is two float32 arrays of $5000 \times 250 \times 250 \times 4$ bytes = 1.25 GB each. The statistical cost is the frozen-noise problem of Section 8.2.

Enhanced (cell 8). Noise is generated inside the input pipeline, per step:

```
train_ds = (
    tf.data.Dataset.from_tensor_slices(train_patches)
    .shuffle(10_000)
    .map(augment_and_noise, num_parallel_calls=tf.data.AUTOTUNE)
    .batch(BATCH)
    .prefetch(tf.data.AUTOTUNE)
)
```

The `map` is applied after `shuffle` and before `batch`, so every epoch re-draws both the augmentation and the corruption; `num_parallel_calls=AUTOTUNE` and `prefetch` overlap this CPU work with GPU compute, so the resampling is effectively free. Stored output: Steps per epoch: 1000.

Note also the explicit cast to `tf.float16` inside `augment_and_noise`, which matches the global mixed-precision policy and avoids a per-batch cast on the critical path.

9.4 Model construction

The baseline (cell 11) and enhanced (cell 10) builders were quoted in full in Sections 6.2–6.4. The line-by-line correspondence is:

Line	Mathematics
Conv2D(64, 3, padding='same', kernel_initializer='he_normal')(inp)	lift to feature space; (5) with $C_{\text{in}} = 1$, $C_{\text{out}} = 64$; He init variance $2/\text{fan_in}$
BatchNormalization()	(10)
Activation('relu')	$\delta(\cdot) = \max(0, \cdot)$
GlobalAveragePooling2D()	squeeze, (21)
Conv2D(C//8, 1, use_bias=False, activation='relu')	$\delta(W_1 z)$, (22)
Conv2D(C, 1, use_bias=False, activation='sigmoid')	$\sigma(W_2 \cdot)$, (22)
Multiply()([x, s])	channel rescale, (23)
Add()([shortcut, x])	residual sum, (8)
Add(dtype='float32')([inp, x])	global skip, $\hat{x} = y + g_\theta(y)$, (4)

The only structural difference between the two builders, beyond `n_blocks`, is the presence of `se_block` and the use of `Input(shape=(None, None, 1))`.

9.5 Loss and metrics

The baseline compiles with a library loss and a library metric:

```

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss='mean_squared_error',
    metrics=['mae'],
)

```

(final_image_denoising.ipynb, cell 13.)

The enhanced model compiles with the custom (27) and a custom MAE that casts to float32 first:

```

def mae_metric(y_true, y_pred):
    return tf.reduce_mean(tf.abs(tf.cast(y_true, tf.float32) - tf.cast(y_pred,
        tf.float32)))

model.compile(
    optimizer=keras.optimizers.Adam(lr_schedule),
    loss=combined_loss,
    metrics=[mae_metric],
)

```

(resnet_enhanced_denoiser.ipynb, cells 9 and 11.)

The explicit `tf.cast(..., tf.float32)` at the top of both functions is necessary, not cosmetic: under `mixed_float16` the labels and predictions arrive as `float16`, whose smallest normal value is about 6×10^{-5} , while the squared/absolute differences at convergence are of order 10^{-4} and the SSIM computation involves products of small local moments. Computing either in `float16` would lose most of the significant digits.

9.6 Training loops

```

history = model.fit(
    train_noisy, train_clean,
    epochs=50,
    batch_size=32,
    validation_split=0.1,
    callbacks=callbacks,
    verbose=1,
)

```

(final_image_denoising.ipynb, cell 14.)

```

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=callbacks,
    verbose=1,
)

```

(*resnet_enhanced_denoiser.ipynb*, cell 13.)

The difference in the first argument — NumPy arrays versus a `tf.data.Dataset` — is the whole of the frozen-versus-resampled-noise distinction. The stored logs record 85 steps per epoch for the baseline ($\lceil 2700/32 \rceil = 85$, after the 10% validation split) and 1,000 for the enhanced model (64,000/64).

The logs also show the two models converging on very different scales, as expected given the different objectives. The baseline’s MSE falls $1.00 \rightarrow 0.0143 \rightarrow 0.0096 \rightarrow 0.0072 \rightarrow 0.0059$ over its first five epochs, with validation MSE reaching 0.00629 by epoch 5; the enhanced model’s combined loss falls $0.174 \rightarrow 0.110 \rightarrow 0.097 \rightarrow 0.091 \rightarrow 0.088$, with validation 0.0846 by epoch 5. The very large epoch-1 values in the baseline (`loss: 1.0016`) reflect the network starting far from the identity in output scale before BN statistics settle.

9.7 Evaluation code

```
test_denoised = np.clip(
    model.predict(test_noisy, batch_size=16, verbose=1).astype(np.float32),
    0.0, 1.0
)

psnr_noisy    = np.mean([PSNR(test_clean[i], test_noisy[i])    for i in
range(len(test_clean))])
psnr_denoised = np.mean([PSNR(test_clean[i], test_denoised[i]) for i in
range(len(test_clean))])
ssim_noisy    = np.mean([SSIM(test_clean[i], test_noisy[i])    for i in
range(len(test_clean))])
ssim_denoised = np.mean([SSIM(test_clean[i], test_denoised[i]) for i in
range(len(test_clean))])
```

(*resnet_enhanced_denoiser.ipynb*, cell 15.)

Two details matter for interpreting Section 10. First, metrics are averaged *per image* rather than computed from an aggregate MSE; by Jensen’s inequality and the convexity of $-\log$, the mean of per-image PSNRs is at least the PSNR of the pooled MSE, so the two conventions are not interchangeable (this is verified numerically in Section 10.3). Second, predictions are clipped to $[0, 1]$ before scoring, which by the projection argument of Section 3.5 can only help — a legitimate but non-neutral post-process that should be stated when comparing with other work.

10 Experiments and Results

10.1 Dataset

Labelled Faces in the Wild (Huang et al., 2007) contains 13,233 face photographs of 5,749 individuals, collected from news sources and detected with the Viola–Jones detector, distributed at 250×250 pixels in the “funneled”/aligned form served by TensorFlow Datasets. It was built for face *verification*, not restoration, which has consequences worth stating plainly:

- The images are **not clean**. They are JPEG-compressed photographs taken with many different cameras under uncontrolled illumination, so the reference used as ground truth already

contains sensor noise, demosaicing artifacts, and DCT blocking.

- The domain is **narrow and highly aligned**: near-frontal, roughly centred faces at a common scale. A denoiser trained here has an unusually strong prior available to it, which is part of why a 6×10^5 -parameter network reaches the numbers below, and part of why those numbers should not be read as comparable to results on BSD68 or Set12.
- There is **no standard denoising split** for LFW, so the protocol below is bespoke and not comparable with published benchmarks.

Both notebooks draw 5,000 images with `dataset.shuffle(7000).take(5000)` (no fixed seed, so the draw differs between notebook runs), convert to grayscale, and normalize to $[0, 1]$.

10.2 Evaluation protocol

	Baseline notebook	Enhanced notebook
Training set	images 0–2999 (full 250×250), 10% split off for validation	patches from images 0–3999 (64,000 crops)
Validation set	10% of the 3,000 training images	16,000 patches from images 4000–4999
Test set	images 3000–4999 (2,000 full images)	images 4000–4999 (1,000 full images)
Test corruption	fixed $\sigma = 0.09$ AWGN, clipped to $[0, 1]$	fixed $\sigma = 0.09$ AWGN, clipped to $[0, 1]$
Metrics	per-image PSNR and SSIM, averaged	per-image PSNR and SSIM, averaged
SSIM implementation	<code>skimage</code> , 7×7 uniform window, <code>data_range=1.0</code>	identical

The head-to-head comparison of Section 10.3 is performed inside the enhanced notebook (cell 16), which loads the baseline’s `best_denoiser.keras` checkpoint and evaluates it on the *same* 1,000 test images and the *same* noise realization as the enhanced model. To that extent the comparison is properly controlled.

10.3 Results

The following are the values printed by the committed notebook outputs (`resnet_enhanced_denoiser.ipynb`, cells 15 and 16), on 1,000 held-out LFW images at $\sigma = 0.09$.

Model	PSNR (dB)	SSIM	Parameters
Noisy input (no processing)	21.42	0.3440	—
Baseline ResNet (5 blocks, MSE)	28.32	0.7241	373,057
Enhanced ResNet (8 RCAB, SSIM + ℓ_1)	33.71	0.9184	604,353

Derived quantities:

Comparison	Δ PSNR	MSE reduction factor	Δ SSIM
Baseline vs noisy	+6.90 dB	$4.90\times$	+0.3801
Enhanced vs noisy	+12.29 dB	$16.94\times$	+0.5744
Enhanced vs baseline	+5.39 dB	$3.46\times$	+0.1943

In absolute terms, the residual root-mean-square error per pixel falls from 0.0849 (noisy, $\approx 21.6/255$) to 0.0384 (baseline, $\approx 9.8/255$) to 0.0206 (enhanced, $\approx 5.3/255$).

Independent corroboration of the baseline number. The baseline notebook evaluates itself on its own held-out 2,000-image split (cell 16), reporting **Test MSE: 0.001538**. Converting via (11),

$$-10 \log_{10}(0.001538) = 28.13 \text{ dB},$$

which agrees closely with the 28.32 dB measured in the enhanced notebook on a different 1,000-image draw. Two consistency observations follow. First, the small gap in the expected direction is what Jensen’s inequality predicts: the enhanced notebook averages *per-image PSNR*, while 28.13 dB is computed from a *pooled MSE*, and since $-\log$ is convex, $\mathbb{E}[-10 \log_{10} \text{MSE}_i] \geq -10 \log_{10} \mathbb{E}[\text{MSE}_i]$. Second, the agreement between an evaluation on the baseline’s own clean split and one on a foreign draw suggests that the cross-notebook contamination discussed in Section 11.2 is small in practice.

Zero-shot transfer to impulse noise. Both notebooks additionally apply the Gaussian-trained model to 5% salt-and-pepper noise, a corruption it has never seen. The enhanced model records (cell 18):

Salt-and-pepper, $f = 0.05$	PSNR (dB)	SSIM
Noisy input	17.75	0.3698
Enhanced ResNet (zero-shot)	22.56	0.5425

The noisy figure is as theory predicts: replacing a fraction f of pixels by 0 or 1 with equal probability gives $\text{MSE} \approx f (\mathbb{E}[x^2] - \mathbb{E}[x] + \frac{1}{2})$, which for typical grayscale face statistics is about 0.015, i.e. ≈ 18 dB. The +4.81 dB the network recovers shows it has learned something genuinely general about face structure rather than a lookup table for Gaussian noise, but 22.56 dB is a poor result in absolute terms: impulse noise violates the Gaussian residual model on which the whole estimator is built, and a plain median filter — the matched estimator for impulse noise — is the appropriate comparison, which the notebooks do not run. This experiment is best read as a demonstration of graceful degradation, not as evidence of broad robustness.

10.4 Interpretation: where the 5.39 dB comes from

The enhanced model differs from the baseline in *six* respects simultaneously: architecture (SE gating), depth (8 blocks versus 5), training data (patches versus full images), noise handling (re-sampled and blind versus frozen and fixed), loss (SSIM + ℓ_1 versus MSE), and schedule (cosine versus plateau-reactive). **No ablation is run, so the attribution below is reasoned, not measured.** Ranked by expected contribution:

1. **Fresh noise every step (Section 8.2, last subsection).** The baseline is fitted to 3,000 frozen (y_i, x_i) pairs for 50 epochs with 3.7×10^5 parameters; the enhanced model sees a new corruption of every patch on every visit. This converts a finite-sample regression into a stochastic one, and it is the difference between fitting $p(x, y)$ from 3,000 samples and from an unbounded stream. On priors from the denoising literature this is the largest single factor.
2. **11.8× more optimization steps per epoch** (1,000 versus 85), on decorrelated mini-batches. Convergence is governed by steps, not epochs.
3. **The loss.** Part of the SSIM gain (+0.194) is a matched-objective effect and should be discounted; the PSNR gain is not, and ℓ_1 -family objectives are known to outperform ℓ_2 even on PSNR (Zhao et al., 2017).
4. **Depth: 37×37 versus 25×25 receptive field**, a $2.2\times$ increase in context area, plus the image-wide context of the SE branch.
5. **Cosine annealing**, worth a few tenths of a decibel in most restoration settings, mostly through the long low-learning-rate tail.
6. **Channel attention itself.** It accounts for 1.4% of the parameters, and Section 6.5 gives a specific reason to expect it to contribute *less* at 250×250 inference than at 64×64 training. Despite the enhanced model being named for it, this is likely the smallest of the six effects.

The honest summary is that this is a *systems* result, not an architecture result: the enhanced notebook is a better-engineered training pipeline that also happens to have attention in it.

10.5 Discrepancy between the README and the notebook outputs

The repository’s `README.md` reports different numbers from the ones the committed notebooks actually printed:

Model	README.md	Notebook output
Noisy input	≈ 20.9 dB / ≈ 0.72	21.42 dB / 0.3440
Baseline	≈ 24 –25 dB / ≈ 0.86	28.32 dB / 0.7241
Enhanced	≈ 26 –28 dB / ≈ 0.91	33.71 dB / 0.9184

The PSNR column is qualitatively consistent in *ordering* but understated by 3–6 dB throughout. The SSIM column shows a suggestive pattern: the README’s noisy SSIM (0.72) is almost exactly the notebook’s *baseline* SSIM (0.7241), and the README’s enhanced SSIM (0.91) matches the notebook’s enhanced SSIM (0.9184) — consistent with a row-shifted transcription, though this is an inference and not established. The README also states that training used 80,000 patches when the 80/20 split means 64,000 were used for gradient updates, and the enhanced notebook’s own header cell says 48,000 patches where the code produces 80,000.

The numbers reported throughout this document are the notebook outputs, since those are reproducible from the committed artifacts. The README should be corrected.

11 Limitations and Failure Cases

11.1 Validation and test sets are the same images

This is the most serious methodological defect, and it is a one-line bug. In `resnet_enhanced_denoiser.ipynb`, cell 7:


```

all_patches = extract_patches(images_clean)      # ALL 5000 images

n_train    = int(0.8 * len(all_patches))
train_patches = all_patches[:n_train]
val_patches  = all_patches[n_train:]

# Hold-out full images for final test (do not use these for patch extraction)
test_clean = images_clean[4000:]

```

The comment states the intent correctly; the code does not implement it. `extract_patches` is called on the *entire* `images_clean` array, and because it emits 16 consecutive patches per image in image order, the first 64,000 patches come from images 0–3999 and the last 16,000 from images 4000–4999. Those last 1,000 images are exactly `test_clean`.

The consequences are precise:

- **Training data is clean.** The 64,000 training patches come from images 0–3999 and never touch a test image. The network’s weights were not fitted on test content. This is the important thing, and it is fine.
- **Model selection is not clean.** `EarlyStopping(restore_best_weights=True)` and `ModelCheckpoint(save_best_only=True)` both select the returned weights by minimizing `val_loss`, which is measured on crops of the test images. The reported test metrics are therefore optimistically biased by a best-of- K -epochs selection performed on the test distribution.

The bias from selecting the best of a few dozen epochs on a 1,000-image distribution is modest — likely a few hundredths of a decibel, not decibels — so the qualitative conclusion survives. But the number is not a clean held-out estimate and should not be quoted as one. The fix is one line: `extract_patches(images_clean[:4000])`, with the 80/20 split then taken within images 0–3999.

11.2 The baseline comparison uses a checkpoint from an independent data draw

Cell 16 of the enhanced notebook loads `best_denoiser.keras`, written by the baseline notebook in an earlier session. Because each notebook independently executes `dataset.shuffle(7000).take(5000)` with no seed, the baseline’s 3,000 training images are a different random subset of LFW’s 13,233 than the enhanced notebook’s test set. Each test image therefore had probability $3000/13233 = 22.7\%$ of appearing in the baseline’s training set, so roughly 227 of the 1,000 test images were plausibly seen by the baseline during training.

This biases the baseline’s 28.32 dB *upward*, which means the enhanced model’s +5.39 dB margin is, if anything, understated. The corroborating evidence in Section 10.3 — that the baseline’s own clean 2,000-image split yields 28.13 dB — suggests the effect is small. But a rigorous comparison would fix a single seed, build one dataset, and train both models on identical splits.

11.3 No ablation, so nothing is attributed

Six changes were made at once (Section 10.4). The experiment establishes that the enhanced *pipeline* is better; it establishes nothing about *why*, and in particular it provides no evidence that channel attention — the feature the model is named for — contributes anything at all. A minimal ablation grid is proposed in Section 12.1.

11.4 Only one noise level is tested, so the blind claim is untested

The enhanced model is *trained* blind over $\sigma \in [0.05, 0.15]$, but it is *evaluated* at the single value $\sigma = 0.09$, near the centre of that range and identical to the baseline’s training level. Nothing in the experiment demonstrates the property that blind training is supposed to deliver, namely robustness across levels. The specific claims that remain unverified are:

- performance at the edges of the training range ($\sigma = 0.05$ and 0.15), where blind models typically lose the most relative to level-specific ones;
- extrapolation beyond the training range ($\sigma = 0.02$, $\sigma = 0.25$), which is where blind denoisers usually fail — and fail asymmetrically, since under-denoising at high σ is more benign than the aggressive over-smoothing that occurs when a model trained at high σ is applied to a nearly clean image;
- the shape of the PSNR-versus- σ curve against a set of level-specific baselines, which is the standard way this trade-off is reported.

Running the existing evaluation cell in a loop over $\sigma \in \{0.02, 0.05, 0.075, 0.09, 0.12, 0.15, 0.20, 0.25\}$ would settle all three at the cost of a few minutes of compute.

11.5 Grayscale only

The RGB-to-luminance conversion discards two thirds of the measurement. Real colour denoising is *easier* per channel than grayscale denoising because noise in R, G, and B is partially independent while the underlying signal is strongly correlated across channels, so a colour denoiser can exploit inter-channel redundancy — this is why CBM3D outperforms channel-wise BM3D substantially. A grayscale result therefore neither transfers to nor bounds colour performance. Extending the notebooks is mechanical (change 1 to 3 in the input and tail convolutions) but changes what is being measured.

11.6 Domain specificity of LFW

The training and test distributions are the same narrow one: aligned, near-frontal, similarly scaled faces. Nothing in this experiment indicates how the models behave on natural scenes, text, medical images, or even on unaligned faces at other scales. A network with a 37×37 receptive field trained exclusively on 250×250 aligned portraits has had the opportunity to learn scale-specific and position-specific regularities (the typical size of an iris, the typical gradient profile of a nostril) that will not hold elsewhere. Reported numbers on LFW are not comparable to the BSD68/Set12/Urban100 numbers used throughout the denoising literature, and should never be quoted alongside them.

Compounding this, as noted in Section 9.2, the LFW “clean” targets are JPEG-compressed and already noisy. The task as posed is “invert synthetic AWGN applied to a mildly degraded reference”, and both the training signal and the evaluation metric treat the reference’s own artifacts as signal to be preserved.

11.7 No real-noise validation

The single largest gap between this project and deployable denoising is that all corruption here is synthetic AWGN. Real sensor noise, after the camera pipeline, is signal-dependent (Poisson–Gaussian), spatially correlated by demosaicing and sharpening, non-Gaussian in the tails, and often has a fixed-pattern component. Plotz and Roth (2017) demonstrated on the Darmstadt Noise Dataset that methods ranked by synthetic-AWGN performance re-order substantially on real

photographs, with several deep models trained on AWGN falling *below* classical BM3D. Guo et al. (2019) reached the same conclusion and addressed it by simulating the whole in-camera pipeline. Nothing in this project’s evaluation predicts its behaviour on a real photograph.

11.8 Metric and protocol caveats

- **Two different SSIMs.** Training optimizes `tf.image.ssim` (11×11 Gaussian window, the Wang et al. definition); reporting uses `skimage.metrics.structural_similarity` with default arguments (7×7 uniform window, sample covariance). The reported absolute SSIM values are therefore not the canonical index and are not comparable to published numbers.
- **Clipping is a free denoiser.** All predictions are clipped to $[0, 1]$ before scoring, which by the projection argument of Section 3.5 can only improve MSE. This is standard and legitimate, but it must be applied to baselines too when comparing.
- **Single seed, no error bars.** Every number is from one training run. There is no estimate of run-to-run variance, so a 0.1 dB difference between configurations could not be distinguished from noise.
- **No classical or literature baselines.** BM3D and a pretrained DnCNN would take minutes to run and would establish whether the numbers here are competitive or merely internally consistent. Without them, the 33.71 dB is uninterpretable in absolute terms.
- **The cosine floor is 10^{-9} , not 10^{-6}** (Section 7.5), because Keras interprets `alpha` as a fraction of the initial rate.

11.9 Architectural caveats

- **Batch normalization contradicts restoration practice.** EDSR, RCAN and NAFNet all remove or replace BN for the reasons in Section 3.4. Its presence here is untested.
- **Train/test resolution mismatch in the attention branch.** Global average pooling over 64^2 during training and 250^2 at inference shifts the distribution of the channel descriptor (Section 6.5).
- **Boundary over-representation.** 80.9% of training-loss pixels are affected by zero-padding, versus 26.7% at test time (Section 8.1).
- **No self-ensembling.** The eight D_4 transforms are free at inference and are not exploited.

12 Future Research Directions

12.1 The ablation that should be run first

Every claim in Section 10.4 is currently reasoning rather than measurement. With a fixed seed and a single shared data split, the following one-factor-at-a-time grid resolves it in roughly a dozen training runs:

Run	Change from baseline	Isolates
A	baseline as committed	reference
B	A + noise resampled every step (still full images, MSE, fixed σ)	frozen-noise effect
C	B + patch training	patches, batch size, step count
D	C + blind σ	blind training cost/benefit

Run	Change from baseline	Isolates
E	D + cosine schedule	schedule
F	E + SSIM + ℓ_1 loss	loss
G	F + 8 blocks (no SE)	depth
H	G + SE = the committed enhanced model	channel attention
I	H with BN removed	BN in restoration
J	H with $r = 16$ instead of 8	SE reduction ratio

Runs G and H are the pair that would, for the first time, actually test the hypothesis the project is named for.

12.2 Real-noise datasets and realistic degradation

The natural next step is to move off synthetic AWGN.

- **SIDD** (Abdelhamed et al., 2018) provides 30,000 real smartphone photographs with high-quality ground truth obtained by careful multi-shot averaging and a statistical estimation procedure; it is now the standard real-noise benchmark, and NAFNet’s headline 40.30 dB is measured on it.
- **DND** (Plotz and Roth, 2017) supplies 50 real scene pairs with a held-out online evaluation server, preventing test-set overfitting.
- **Realistic synthesis.** Where paired real data is unavailable, CBDNet’s approach (Guo et al., 2019) — simulate Poisson–Gaussian noise in the raw domain, then push it through demosaicing, colour correction, and tone mapping — produces training pairs whose noise statistics actually resemble a camera’s.

12.3 Self-supervised and unpaired training

Clean references are often impossible to obtain (fluorescence microscopy, astronomy, real photography). Three lines remove that requirement:

- **Noise2Noise** (Lehtinen et al., 2018) shows that training on pairs of *independently noisy* observations of the same scene converges to the same optimum as clean-target training, because for a zero-mean corruption $\mathbb{E}[y' \mid x] = x$, so regressing onto y' has the same minimizer as regressing onto x — only the gradient variance differs.
- **Noise2Void** (Krull et al., 2019) removes even that requirement using blind-spot networks: the receptive field is masked at the centre pixel, so the network cannot copy the noise it is asked to predict and must instead infer the pixel from its context. This requires only pixel-wise independent noise.
- **Neighbor2Neighbor** (Huang et al., 2021) constructs training pairs by sub-sampling adjacent pixels from a single noisy image, avoiding the architectural constraint of blind-spot networks.

Applying Noise2Void to LFW would be an especially clean demonstration here, since the “clean” LFW references are themselves imperfect (Section 9.2) and a self-supervised method sidesteps that problem entirely.

12.4 Colour, and joint restoration

Extending to three channels and exploiting inter-channel correlation is the cheapest available improvement. Beyond that lies joint handling of the degradations LFW actually contains — JPEG artifacts, mild blur — which DnCNN already showed a single network can address when trained on a mixture (Zhang et al., 2017), and which is the subject of the “all-in-one” restoration literature.

12.5 Modern architectures

Three concrete upgrades, in increasing order of cost:

- **Remove BN, add residual scaling** (EDSR-style), the change most likely to yield an immediate gain at zero parameter cost.
- **Restormer-style transposed attention** (Zamir et al., 2022): replace the SE gate with attention computed across channels, giving a full $C \times C$ content-dependent mixing matrix instead of a diagonal one, at cost linear in the number of pixels. This is the natural generalization of exactly the mechanism already present.
- **NAFNet blocks** (Chen et al., 2022): SimpleGate plus simplified channel attention, in a UNet-shaped backbone. NAFNet is the strongest accuracy-per-FLOP option currently available for denoising and is architecturally *simpler* than what is implemented here.

12.6 Diffusion and plug-and-play

Given Tweedie’s formula (4), the trained denoiser here is already an approximate score model for LFW faces at $\sigma \in [0.05, 0.15]$. Two things follow that would cost little to try:

- Use the network as the proximal step in a Plug-and-Play (Venkatakrishnan et al., 2013) solver for a harder inverse problem on faces — inpainting, deblurring, or $4\times$ super-resolution — with no retraining.
- Train it across a wider σ schedule and use it as a sampler in the manner of Kadkhodaie and Simoncelli (2021), to generate faces and, more usefully, to produce *multiple plausible reconstructions* of a single noisy input — which makes the posterior uncertainty visible rather than averaging it into blur.

12.7 Perceptual objectives and honest metrics

Blau and Michaeli (2018) proved that distortion and perceptual quality cannot be optimized simultaneously past a certain frontier, so the choice of objective should be made explicitly rather than by default. Options, all applicable here:

- **Perceptual (feature) losses** on the activations of a pretrained network, and **LPIPS** (Zhang et al., 2018c) as an evaluation metric that correlates far better with human judgement than PSNR or SSIM.
- **Adversarial losses** (Ledig et al., 2017), which move the output onto the natural-image manifold at a measurable PSNR cost — appropriate when the consumer is a human, inappropriate when it is a measurement.
- **No-reference metrics** such as NIQE (Mittal et al., 2013) for the real-noise setting where no ground truth exists.
- **MS-SSIM** (Wang et al., 2003) in place of single-scale SSIM in the loss, which Zhao et al. (2017) found superior in exactly the mixture used here.

12.8 Practical engineering

Test-time self-ensembling over the eight D_4 transforms; computing the training loss only on the valid interior of each patch to remove the boundary over-representation of Section 8.1; larger training crops (or tiled inference at 64×64) to close the global-pooling mismatch of Section 6.5; and fixed seeds with multiple runs so that differences of a few tenths of a decibel become interpretable.

13 Conclusion

The repository implements, correctly and compactly, a design lineage that runs from He et al.’s residual block through Zhang et al.’s residual denoising to Hu et al.’s channel attention, and it demonstrates a large measured improvement: $21.42 \rightarrow 28.32 \rightarrow 33.71$ dB PSNR and $0.344 \rightarrow 0.724 \rightarrow 0.918$ SSIM across noisy input, baseline, and enhanced model on 1,000 held-out LFW images at $\sigma = 0.09$. The parameter accounting, the receptive-field arithmetic, and the noisy-input PSNR all check out exactly against theory, which is a good sign that the implementation does what it is described as doing.

Three things temper the result. The improvement is a *pipeline* improvement, of which channel attention is very likely the smallest component and is entirely unmeasured. The validation set is the test set, so the headline number carries a model-selection bias. And the setting — grayscale, synthetic AWGN, one noise level, one narrow image domain, no external baselines — is narrow enough that the absolute numbers carry no information about competitiveness with the literature. Each of these is addressable with the changes set out in Sections 11 and 12; the ablation grid of Section 12.1 and a σ -sweep are the two that would most improve the work’s scientific value per unit of effort.

14 References

1. Abdelhamed, A., Lin, S., and Brown, M. S. (2018). A High-Quality Denoising Dataset for Smartphone Cameras. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1692–1700. DOI: 10.1109/CVPR.2018.00182.
2. Blau, Y. and Michaeli, T. (2018). The Perception-Distortion Tradeoff. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6228–6237. arXiv:1711.05189.
3. Buades, A., Coll, B., and Morel, J.-M. (2005). A Non-Local Algorithm for Image Denoising. *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 2, pp. 60–65. DOI: 10.1109/CVPR.2005.38.
4. Buades, A., Coll, B., and Morel, J.-M. (2005b). A Review of Image Denoising Algorithms, with a New One. *Multiscale Modeling and Simulation (SIAM Interdisciplinary Journal)*, 4(2), pp. 490–530. DOI: 10.1137/040616024.
5. Burger, H. C., Schuler, C. J., and Harmeling, S. (2012). Image Denoising: Can Plain Neural Networks Compete with BM3D? *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2392–2399. DOI: 10.1109/CVPR.2012.6247952.
6. Chen, H., Wang, Y., Guo, T., Xu, C., Deng, Y., Liu, Z., Ma, S., Xu, C., Xu, C., and Gao, W. (2021). Pre-Trained Image Processing Transformer (IPT). *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12299–12310. arXiv:2012.00364.

7. Chen, L., Chu, X., Zhang, X., and Sun, J. (2022). Simple Baselines for Image Restoration (NAFNet). *European Conference on Computer Vision (ECCV)*, pp. 17–33. arXiv:2204.04676. DOI: 10.1007/978-3-031-20071-7_2.
8. Chen, Y. and Pock, T. (2017). Trainable Nonlinear Reaction Diffusion: A Flexible Framework for Fast and Effective Image Restoration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6), pp. 1256–1272. DOI: 10.1109/TPAMI.2016.2596743. arXiv:1508.02848.
9. Dabov, K., Foi, A., Katkovnik, V., and Egiazarian, K. (2007). Image Denoising by Sparse 3-D Transform-Domain Collaborative Filtering (BM3D). *IEEE Transactions on Image Processing*, 16(8), pp. 2080–2095. DOI: 10.1109/TIP.2007.901238.
10. Donoho, D. L. (1995). De-noising by Soft-Thresholding. *IEEE Transactions on Information Theory*, 41(3), pp. 613–627. DOI: 10.1109/18.382009.
11. Donoho, D. L. and Johnstone, I. M. (1994). Ideal Spatial Adaptation by Wavelet Shrinkage. *Biometrika*, 81(3), pp. 425–455. DOI: 10.1093/biomet/81.3.425.
12. Efron, B. (2011). Tweedie’s Formula and Selection Bias. *Journal of the American Statistical Association*, 106(496), pp. 1602–1614. DOI: 10.1198/jasa.2011.tm1181.
13. Elad, M. and Aharon, M. (2006). Image Denoising via Sparse and Redundant Representations over Learned Dictionaries. *IEEE Transactions on Image Processing*, 15(12), pp. 3736–3745. DOI: 10.1109/TIP.2006.881969.
14. Guo, S., Yan, Z., Zhang, K., Zuo, W., and Zhang, L. (2019). Toward Convolutional Blind Denoising of Real Photographs (CBDNet). *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1712–1722. arXiv:1807.04686.
15. He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *IEEE International Conference on Computer Vision (ICCV)*, pp. 1026–1034. arXiv:1502.01852.
16. He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep Residual Learning for Image Recognition. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778. arXiv:1512.03385. DOI: 10.1109/CVPR.2016.90.
17. He, K., Zhang, X., Ren, S., and Sun, J. (2016b). Identity Mappings in Deep Residual Networks. *European Conference on Computer Vision (ECCV)*, pp. 630–645. arXiv:1603.05027.
18. Ho, J., Jain, A., and Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, pp. 6840–6851. arXiv:2006.11239.
19. Hu, J., Shen, L., and Sun, G. (2018). Squeeze-and-Excitation Networks. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7132–7141. arXiv:1709.01507. DOI: 10.1109/CVPR.2018.00745.
20. Huang, G. B., Ramesh, M., Berg, T., and Learned-Miller, E. (2007). Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments. *Technical Report 07-49*, University of Massachusetts, Amherst.
21. Huang, T., Li, S., Jia, X., Lu, H., and Liu, J. (2021). Neighbor2Neighbor: Self-Supervised Denoising from Single Noisy Images. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14781–14790. arXiv:2101.02824.

22. Ioffe, S. and Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *International Conference on Machine Learning (ICML)*, pp. 448–456. arXiv:1502.03167.
23. Jain, V. and Seung, S. (2008). Natural Image Denoising with Convolutional Networks. *Advances in Neural Information Processing Systems (NIPS)*, 21, pp. 769–776.
24. Kadkhodaie, Z. and Simoncelli, E. P. (2021). Stochastic Solutions for Linear Inverse Problems using the Prior Implicit in a Denoiser. *Advances in Neural Information Processing Systems (NeurIPS)*, 34. arXiv:2007.13640.
25. Kingma, D. P. and Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.
26. Krull, A., Buchholz, T.-O., and Jug, F. (2019). Noise2Void — Learning Denoising from Single Noisy Images. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2129–2137. arXiv:1811.10597.
27. Ledig, C., Theis, L., Huszar, F., Caballero, J., Cunningham, A., Acosta, A., Aitken, A., Tejani, A., Totz, J., Wang, Z., and Shi, W. (2017). Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network (SRGAN). *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4681–4690. arXiv:1609.04802.
28. Lehtinen, J., Munkberg, J., Hasselgren, J., Laine, S., Karras, T., Aittala, M., and Aila, T. (2018). Noise2Noise: Learning Image Restoration without Clean Data. *International Conference on Machine Learning (ICML)*, pp. 2965–2974. arXiv:1803.04189.
29. Levin, A. and Nadler, B. (2011). Natural Image Denoising: Optimality and Inherent Bounds. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2833–2840. DOI: 10.1109/CVPR.2011.5995309.
30. Liang, J., Cao, J., Sun, G., Zhang, K., Van Gool, L., and Timofte, R. (2021). SwinIR: Image Restoration Using Swin Transformer. *IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pp. 1833–1844. arXiv:2108.10257.
31. Lim, B., Son, S., Kim, H., Nah, S., and Lee, K. M. (2017). Enhanced Deep Residual Networks for Single Image Super-Resolution (EDSR). *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 1132–1140. arXiv:1707.02921.
32. Loshchilov, I. and Hutter, F. (2017). SGDR: Stochastic Gradient Descent with Warm Restarts. *International Conference on Learning Representations (ICLR)*. arXiv:1608.03983.
33. Mallat, S. G. (1989). A Theory for Multiresolution Signal Decomposition: The Wavelet Representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(7), pp. 674–693. DOI: 10.1109/34.192463.
34. Mittal, A., Soundararajan, R., and Bovik, A. C. (2013). Making a “Completely Blind” Image Quality Analyzer (NIQE). *IEEE Signal Processing Letters*, 20(3), pp. 209–212. DOI: 10.1109/LSP.2012.2227726.
35. Miyasawa, K. (1961). An Empirical Bayes Estimator of the Mean of a Normal Population. *Bulletin of the International Statistical Institute*, 38, pp. 181–188.
36. Perona, P. and Malik, J. (1990). Scale-Space and Edge Detection Using Anisotropic Diffusion. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(7), pp. 629–639. DOI:

10.1109/34.56205.

37. Plotz, T. and Roth, S. (2017). Benchmarking Denoising Algorithms with Real Photographs (DND). *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1586–1595. arXiv:1707.01313.
38. Robbins, H. (1956). An Empirical Bayes Approach to Statistics. *Proceedings of the Third Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, pp. 157–163. University of California Press.
39. Rudin, L. I., Osher, S., and Fatemi, E. (1992). Nonlinear Total Variation Based Noise Removal Algorithms. *Physica D: Nonlinear Phenomena*, 60(1–4), pp. 259–268. DOI: 10.1016/0167-2789(92)90242-F.
40. Santurkar, S., Tsipras, D., Ilyas, A., and Madry, A. (2018). How Does Batch Normalization Help Optimization? *Advances in Neural Information Processing Systems (NeurIPS)*, 31, pp. 2483–2493. arXiv:1805.11604.
41. Szegedy, C., Ioffe, S., Vanhoucke, V., and Alemi, A. (2017). Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning. *AAAI Conference on Artificial Intelligence*, pp. 4278–4284. arXiv:1602.07261.
42. Tomasi, C. and Manduchi, R. (1998). Bilateral Filtering for Gray and Color Images. *IEEE International Conference on Computer Vision (ICCV)*, pp. 839–846. DOI: 10.1109/ICCV.1998.710815.
43. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NIPS)*, 30, pp. 5998–6008. arXiv:1706.03762.
44. Venkatakrisnan, S. V., Bouman, C. A., and Wohlberg, B. (2013). Plug-and-Play Priors for Model Based Reconstruction. *IEEE Global Conference on Signal and Information Processing (GlobalSIP)*, pp. 945–948. DOI: 10.1109/GlobalSIP.2013.6737048.
45. Wang, Z. and Bovik, A. C. (2009). Mean Squared Error: Love It or Leave It? A New Look at Signal Fidelity Measures. *IEEE Signal Processing Magazine*, 26(1), pp. 98–117. DOI: 10.1109/MSP.2008.930649.
46. Wang, Z., Bovik, A. C., Sheikh, H. R., and Simoncelli, E. P. (2004). Image Quality Assessment: From Error Visibility to Structural Similarity. *IEEE Transactions on Image Processing*, 13(4), pp. 600–612. DOI: 10.1109/TIP.2003.819861.
47. Wang, Z., Simoncelli, E. P., and Bovik, A. C. (2003). Multiscale Structural Similarity for Image Quality Assessment. *Thirty-Seventh Asilomar Conference on Signals, Systems and Computers*, vol. 2, pp. 1398–1402. DOI: 10.1109/ACSSC.2003.1292216.
48. Zamir, S. W., Arora, A., Khan, S., Hayat, M., Khan, F. S., and Yang, M.-H. (2022). Restormer: Efficient Transformer for High-Resolution Image Restoration. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5718–5729. arXiv:2111.09881.
49. Zhang, K., Zuo, W., Chen, Y., Meng, D., and Zhang, L. (2017). Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising (DnCNN). *IEEE Transactions on Image Processing*, 26(7), pp. 3142–3155. DOI: 10.1109/TIP.2017.2662206. arXiv:1608.03981.

50. Zhang, K., Zuo, W., and Zhang, L. (2018b). FFDNet: Toward a Fast and Flexible Solution for CNN-Based Image Denoising. *IEEE Transactions on Image Processing*, 27(9), pp. 4608–4622. DOI: 10.1109/TIP.2018.2839891. arXiv:1710.04026.
51. Zhang, R., Isola, P., Efros, A. A., Shechtman, E., and Wang, O. (2018c). The Unreasonable Effectiveness of Deep Features as a Perceptual Metric (LPIPS). *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 586–595. arXiv:1801.03924.
52. Zhang, Y., Li, K., Li, K., Wang, L., Zhong, B., and Fu, Y. (2018). Image Super-Resolution Using Very Deep Residual Channel Attention Networks (RCAN). *European Conference on Computer Vision (ECCV)*, pp. 294–310. arXiv:1807.02758. DOI: 10.1007/978-3-030-01234-2_18.
53. Zhao, H., Gallo, O., Frosio, I., and Kautz, J. (2017). Loss Functions for Image Restoration with Neural Networks. *IEEE Transactions on Computational Imaging*, 3(1), pp. 47–57. DOI: 10.1109/TCI.2016.2644865. arXiv:1511.08861.
54. TensorFlow Datasets. LFW dataset catalog entry. <https://www.tensorflow.org/datasets/catalog/lfw>